Vivekanand Education Society's Institute of Technology

An Autonomous Institute Affiliated to University of Mumbai

Hashu Advani Memorial Complex, Collector Colony, Chembur East, Mumbai - 400074

Department of Artificial Intelligence and Data Science

A Major Project Report on

Mechanical Verification of Reasoning–Code Alignment in LLM–Driven AutoML Pipelines

Submitted in partial fulfillment of the requirements
for the award of the degree of

Bachelor of Engineering

in

Artificial Intelligence and Data Science

Submitted by

Samarth Adsare	02
Kinnshuk Bhaduri	06
Ojas Godambe	18
Sarvesh Rakte	44

Under the Guidance of

Mr. Amit Singh
Project Guide

Academic Year 2026–2027

Certificate

This is to certify that the project entitled **"Mechanical Verification of Reasoning–Code Alignment in LLM–Driven AutoML Pipelines"** has been carried out by **Samarth Adsare**, **Kinnshuk Bhaduri**, **Ojas Godambe**, and **Sarvesh Rakte** under the guidance of **Mr. Amit Singh** in partial fulfillment of the requirements for the award of the degree of Bachelor of Engineering in Artificial Intelligence and Data Science during the academic year 2026–2027.

Mr. Amit Singh	**Dr. Anjali Yeole**	**Dr. J.M. Nair**
Project Guide	Head of Department	Principal

Date: _____ Place: Mumbai

Declaration

We hereby declare that the project entitled **"Mechanical Verification of Reasoning–Code Alignment in LLM–Driven AutoML Pipelines"** is our original work carried out under the guidance of **Mr. Amit Singh**. The work presented in this report has not been submitted elsewhere for the award of any degree or diploma, except where due acknowledgement has been made.

Name	**Signature**
Samarth Adsare	_____
Kinnshuk Bhaduri	_____
Ojas Godambe	_____
Sarvesh Rakte	_____

Acknowledgement

We would like to express our sincere gratitude to our project guide, **Mr. Amit Singh**, for his valuable guidance, continuous support, constructive suggestions, and encouragement throughout the development of our Major Project. His expertise and feedback have helped us improve our understanding of the subject and complete this project successfully.

We would like to express our sincere thanks to **Dr. Anjali Yeole**, Head of the Department, and **Dr. Sangeeta Oswal**, Deputy Head of the Department, of Artificial Intelligence and Data Science, for providing the necessary guidance, encouragement, and academic support throughout the project.

We are grateful to **Dr. J. M. Nair**, Principal, and **Dr. Greisha Bhatia**, Vice Principal, of Vivekanand Education Society's Institute of Technology (VESIT), for providing the facilities and academic environment required to carry out this project.

We also extend our gratitude to the Department of Artificial Intelligence and Data Science, VESIT, for providing the resources and opportunities that supported the successful completion of this work.

We would also like to thank our classmates, friends, and all those who directly or indirectly contributed to this project through their valuable suggestions, cooperation, and encouragement.

Finally, we express our heartfelt gratitude to our parents and families for their constant motivation, patience, and support throughout the project.

Samarth Adsare
Kinnshuk Bhaduri
Ojas Godambe
Sarvesh Rakte

Abstract

Large Language Models (LLMs) are increasingly used as the reasoning and code–generating backbone of Automated Machine Learning (AutoML) pipelines. In such systems an LLM first produces a natural–language or semi–structured plan that describes how a predictive model will be built, and then emits executable code that is expected to realise exactly that plan. Existing evaluations of these pipelines check that the generated program compiles, runs to completion, and reaches an acceptable score on some downstream metric. None of these checks answers a strictly stronger question: does the emitted code mechanically implement the plan for which the LLM was prepared?

This project formulates the above question as a stand–alone verification problem that we call *reasoning–code alignment* verification. The project has three parts, which the report is careful to keep distinct. First, as a *design* contribution, we specify a verification framework that lifts an AutoML plan to a set of atomic typed claims (a Reasoning Intermediate Representation, RIR), lowers the generated Python program to a directed attributed multigraph fusing abstract syntax, dataflow and library–level semantics (a Program Intermediate Representation, PIR), and maps the two so that each claim receives a three–valued verdict (ALIGNED, MISALIGNED or UNVERIFIABLE) and the pipeline a four–valued one that adds PARTIALLY ALIGNED, with an accompanying AutoML–specific mismatch taxonomy and a coverage, weighted–alignment, critical–mismatch and localisation metric suite.

Second, as an *implementation* contribution, we build a working end–to–end LLM–driven AutoML stack — five FastAPI microservices and a Streamlit front–end, a Redis–backed job queue and worker, a configurable LLM runner (Ollama in the reported runs), a bounded subprocess executor, and a PostgreSQL store with artefacts on the local filesystem — that generates scikit–learn pipelines under three prompt conditions (B0 naive, B1 schema, B2 meta–feature–guided) and runs a first mechanical faithfulness verifier on every B2 run, checking that each cited action appears in the generated code and that each cited dataset statistic matches the real meta–feature. Because this trace is printed by the generated code at run time, it is a post–hoc self–report rather than a plan captured before the code; the check therefore tests code–trace consistency, not independently grounded intent.

Third, as an *empirical* contribution, we evaluate the stack on a controlled 270–cell sweep (6 tabular datasets $\times$ 3 conditions $\times$ 3 model backends $\times$ 5 seeds). The naive and schema conditions are completely reliable (100 % runnable code),

whereas the meta–feature condition is markedly less reliable (70.0 % of runs executed and produced a score; 62.2 % also passed the faithfulness check) and roughly 5–27× more expensive in prompt tokens, without any statistically significant improvement in predictive score (paired Wilcoxon over executed runs, all $p > 0.09$). Crucially, the faithfulness verifier flagged seven B2 runs that executed successfully and produced good scores but whose stated reasoning did not match their code — a failure mode that execution– and performance–based checks cannot detect. The richer graph–based verifier, the independent pre–code reasoning capture and the baseline comparison are designed but not implemented or evaluated in this project; the report marks these as future work rather than completed results.

Keywords: automated machine learning, large language models, LLM agents, code generation, program analysis, mechanical verification, reasoning–code alignment, reasoning faithfulness.

Contents

Certificate i

Declaration ii

Acknowledgement iii

Abstract iv

1 Introduction 1
- 1.1 Background . . . 1
- 1.2 Problem Statement . . . 2
- 1.3 Motivation . . . 2
- 1.4 Objectives . . . 3
- 1.5 Scope . . . 4
- 1.6 Contributions . . . 5
- 1.7 Organization of the Report . . . 6

2 Literature Review 8
- 2.1 Introduction . . . 8
- 2.2 Existing Systems . . . 8
  - 2.2.1 LLM–Based AutoML Pipelines . . . 8
  - 2.2.2 Code and Reasoning Verification . . . 9
  - 2.2.3 Formal, Typed and Graph–Based Verification of Reasoning . 9
- 2.3 Review of Research Papers . . . 10
- 2.4 Research Gap . . . 11
- 2.5 Summary . . . 12

3 Proposed Methodology 14
- 3.1 Overview . . . 14
- 3.2 System Requirements . . . 15
  - 3.2.1 Hardware Requirements . . . 15
  - 3.2.2 Software Requirements . . . 15
- 3.3 Proposed Approach . . . 16
  - 3.3.1 Task Specification . . . 16
  - 3.3.2 Reasoning Intermediate Representation (RIR) . . . 16
  - 3.3.3 Code Generation and Program IR . . . 17
- 3.4 Algorithms / Models . . . 18

3.4.1 Semantic Extraction, Mapping and Verdicts 18
3.4.2 Alignment Relation 18
3.5 Evaluation Metrics 19
3.5.1 Semantic Equivalence 20
3.5.2 Worked Example 20
3.6 Mismatch Taxonomy 22
3.7 Baselines 23
3.8 Data Collection and Preprocessing 24
3.9 Summary 24

4 System Design **26**
4.1 System Architecture 26
4.1.1 Ingestion and Generation Tier 26
4.1.2 Run and Verification Tier 26
4.1.3 Analysis Tier 28
4.2 System Workflow 28
4.3 Use Case Diagram 29
4.4 Data Flow 29
4.5 Database Design 30
4.6 Module Description 33

5 Implementation **35**
5.1 Implementation Status 35
5.2 Development Environment 36
5.3 Implemented Services 37
5.3.1 Data Service 38
5.3.2 Meta-Feature Service 38
5.3.3 Prompt Conditions: B0, B1, B2 39
5.3.4 Generation Service and RQ Worker 40
5.3.5 Bounded Subprocess Executor and Error Taxonomy 41
5.3.6 Implemented Verifier (Prototype) 42
5.3.7 Persistence and Analysis 43
5.4 Proposed Components (Designed, Not Yet Implemented) 43
5.5 Algorithm: Implemented vs Proposed 43
5.6 Security and Privacy 46
5.7 Summary 46

6 Results and Discussion **47**
6.1 Experimental Setup 47
6.1.1 Datasets 48

6.2 Pipeline-Level Results . . . 48
6.3 Evaluation of the Research Questions . . . 50
6.3.1 RQ1 — Representation . . . 50
6.3.2 RQ2 — Discrimination (and H1, H2) . . . 51
6.3.3 RQ3 — Categories . . . 51
6.3.4 RQ4 — Variation and Predictive Score . . . 52
6.3.5 RQ5 — Localisation . . . 53
6.4 Hypothesis Summary . . . 54
6.5 Comparison with Baselines (Planned) . . . 54
6.6 Discussion . . . 55
6.7 Threats to Validity . . . 55
6.8 Summary . . . 56

7 Conclusion and Future Scope 57
7.1 Conclusion . . . 57
7.2 Limitations . . . 58
7.3 Future Scope . . . 59
7.4 Final Remarks . . . 60

References 61

A Appendix 65
A.1 Publication . . . 65
A.1.1 Publication Details . . . 65
A.1.2 Publication Evidence . . . 65
A.2 Patent / Copyright . . . 66
A.2.1 Intellectual Property Details . . . 66
A.2.2 Evidence . . . 66
A.3 Open Source Contribution . . . 66
A.3.1 Evidence . . . 67
A.4 Data Contribution . . . 67
A.4.1 Evidence . . . 68
A.5 Research Grant Proposal . . . 68
A.5.1 Evidence . . . 68
A.6 Research Commercialization Efforts . . . 68
A.6.1 Startup Formation . . . 68
A.6.2 Technology Transfer . . . 69
A.6.3 Industry Collaboration . . . 69
A.6.4 Evidence . . . 69
A.7 Research Conclave . . . 70

A.7.1 Evidence 70
A.8 SDG Goals Achieved 70
A.9 Course Outcomes, PO and PSO Mapping 71
A.9.1 Course Outcomes 71
A.9.2 Project Contribution to Program Outcomes 72
A.9.3 Project Contribution to Programme Specific Outcomes . . . 74
A.10 Additional Evidence 80

List of Figures

3.1 Worked RIR–PIR claim–to–code mapping for a single `RobustScaler` decision on the `Fare` column. 22

4.1 System architecture: (a) the runtime flow of one run, and (b) the deployed components and their connections. 27
4.2 Principal use cases of the system. 29
4.3 Entity–relationship diagram of the PostgreSQL schema. 31
4.4 ORM model for the `datasets` table. 31
4.5 ORM model for the `meta_features` table; the `UniqueConstraint` enforces one profile per dataset. 32
4.6 ORM model for the `run_results` table; note the nullable B2–only `reasoning_trace` and `verification_report` JSON columns and the token counters. 32
4.7 ORM model for the `sweep_jobs` table. 33

5.1 Data–service ingestion path (dataset registration and split). 38
5.2 Meta–feature helpers: a column is categorical if boolean, low–cardinality integer, or non–numeric. 39
5.3 The B2 prompt builder assembling the schema, meta–feature profile, decision rules and mandatory reasoning–trace sections. 40
5.4 Enqueuing one sweep cell as a Redis Queue job; the job runs `jobs.run_cell` on a worker. 41
5.5 The rule–based execution–error classifier (`classify_error`); the first matching rule wins. 42
5.6 Implemented boolean faithfulness check (`verify_reasoning` on `main`): for each decision, action–presence in the AST name set and meta–feature value grounding, reduced to a single boolean verdict (an all–faithful run passes; otherwise `reasoning_unfaithful`). . . . 44
5.7 Proposed full RIR–PIR verifier (designed in Chapter 3, *not* implemented): per–claim three–valued verdicts over presence, semantics, parameters, ordering and dependency, aggregated with criticality gating into coverage, weighted alignment, critical–mismatch rate, localisation and a four–valued pipeline verdict. 45

6.1 Success rate by condition and backend. 50
6.2 Distribution of B2 failure categories. 52

6.3 Predictive score by condition. 53

List of Tables

2.1 Representative approaches relative to the proposed verification layer. 11

3.1 Proposed taxonomy of AutoML reasoning–code misalignment. . . . 23
3.2 Baseline capabilities. 24

5.1 Implementation status of each system component. 36
5.2 Backend services, front–ends and their host ports. 37

6.1 The six–dataset panel (all binary classification). 48
6.2 Per–condition results over all 90 cells each. *Verified yield* = runs that were runnable *and* passed the B2 faithfulness check; *raw execution* = runs that produced a pipeline and a score. B0/B1 have no faithfulness gate, so the two coincide and the verifier is not applicable. Mean score is over executed runs; mean iters over successful runs. 48
6.3 B2 outcomes per backend (30 cells each): verified successes, runs that executed and scored but failed the faithfulness check, other failures, and the resulting raw execution count (raw execution = verified + executed-but-unfaithful). 49
6.4 Per–backend × condition: verified success (`ok` = verified yield; raw execution is in Table 6.3), iterations (averaged over all 30 cells, including failed attempts) and mean token counts (over the cells that recorded usage — 30 per row except nemotron–nano B2, where only 14 of 30 did, the rest failing before a usage record was written). 49
6.5 Mean `balanced_accuracy` per dataset × condition over *executed* runs; `n` is the number that executed out of 15 (for B2 this includes runs that produced a score but failed the faithfulness check). 50
6.6 Failure categories for the 34 B2 cells that did not meet the end–to–end success definition (B0/B1 had none). `reasoning_unfaithful` marks a run that executed and scored but was rejected by the verifier, not an execution failure. 52

6.7 Paired Wilcoxon signed–rank on test score, over matched cells where both conditions produced a score. **n** is the number of *discordant* pairs (those with a non–zero score difference) used by the signed–rank test; the remaining matched pairs scored identically. Median Δ (second condition minus first) is taken over those discordant pairs. $\mathbf{r} = |z|/\sqrt{n}$ is the rank effect size; all three are small (≤ 0.21). B2 includes every run that produced a score, whether or not the verifier accepted it. 53
6.8 Status of the four hypotheses against the 270–cell run. 54
6.9 Structure of the planned baseline comparison. Entries are left blank pending the seeded–mutation corpus. 54

A.1 SDG Goals Addressed by the Project 71
A.2 Project Contribution to Program Outcomes 73
A.3 Project Contribution to Programme Specific Outcomes 74
A.4 CO–PO Mapping 75
A.5 CO–PSO Mapping 76

Chapter 1

Introduction

1.1 Background

Automated Machine Learning (AutoML) is the subfield of machine learning that aims to automate the end–to–end construction of predictive systems, including data cleaning, feature engineering, model selection, hyper–parameter optimisation and model evaluation. Early AutoML frameworks such as Auto–Sklearn [1] and AMLB [2] search over a fixed, hand–designed space of pipelines and return the best configuration found under a budget. The user describes the task through a structured API or a configuration file, and the engine returns a trained model and its measured performance.

The arrival of capable Large Language Models (LLMs) has reshaped this picture. A user can now describe the task, the data constraints and the deployment requirements in free–form natural language, and an LLM–driven agent can decide which features to construct, which model family to try, how to split the data, and how to evaluate the result. New systems embed this behaviour into full pipelines, including feature engineering agents [3], data–interpreter–style systems [4], knowledge–driven data–science agents [5], tree–search enhanced agents [6] and multi–agent full–pipeline AutoML frameworks [7]. A recent survey on LLMs for building and optimising ML workflows [8] catalogues this growing landscape and highlights the open problems that follow from it.

A consequence of this shift is that the automated pipeline now produces an artifact that traditional AutoML did not surface: a *plan*, or natural–language justification, that explains what the system intends to do before it writes a single line of code. An LLM may, for example, state that it will impute missing values with the median, use a stratified train/validation split, standardise the numerical columns using training statistics only, fit a logistic model under a class–imbalance constraint, and report the macro F1–score for model selection. The downstream coding agent then generates the executable pipeline that is supposed to realise this plan.

Existing checks tell us a great deal about such pipelines: syntax and type checks reject malformed programs, sandboxed execution surfaces runtime errors, tests exercise selected behaviour, standard AutoML evaluation measures predictive quality and cost, and execution feedback can even be fed back for self–repair [9,10]. What none of them establishes is whether the program that runs is the one the

preceding plan actually intended.

1.2 Problem Statement

The project targets the following problem.

> Given an AutoML plan produced by an LLM reasoning agent and a Python pipeline produced by a coding agent, mechanically determine whether the pipeline implements that plan, and if it does not, identify precisely which claims of the plan are missing, contradicted, mis–parameterised, mis–ordered or merely unverifiable from the available evidence.

In the terminology of this project, this is the *reasoning–code alignment* problem. It is distinct from reasoning correctness (is the plan itself sound?), from code correctness (does the code satisfy a code specification?), from execution correctness (does the code run and produce an output?) and from predictive performance (does the code reach an acceptable score?). A plan can be sound and still be realised by an incorrect program; a program can be well–engineered and still implement a different plan from the one presented to the user; two programs that look lexically different may implement the same plan; and a program that reaches a strong score may still have replaced the intended estimator or quietly dropped a required pre–processing step.

Formally, if T is the task specification, R the reasoning artifact, C the corresponding source code, E the execution trace and Y the achieved performance, then current pipelines check variants of

$$Q_R : \text{Is } R \text{ a justifiable strategy for } T?, \quad Q_C : \text{Does } C \text{ satisfy a code specification?},$$

$$Q_E : C \xrightarrow{\text{execute}} E \text{ with satisfactory } Y,$$

whereas the question of interest here is strictly a fourth one,

$$Q_A : \text{Does } C \text{ perform the actions specified in } R?$$

None of Q_R, Q_C, Q_E implies Q_A, which is why reasoning–code alignment deserves its own verification layer.

1.3 Motivation

Three considerations drive the project.

Plans are now control artifacts, not just explanations. In multi–agent AutoML systems the plan is what a human reviewer or a downstream module approves; it is also what the user is shown as the justification for a deployment decision. If the generated code silently diverges from the plan, every subsequent operational action is taken on a false premise. A pipeline that is approved on the basis of a careful, class–imbalance–aware plan but is actually realised by a random split with a different estimator may still produce a credible accuracy number, and that number can mask a systematic failure on the minority class.

Execution evidence is weaker than it looks. It has already been established that executable code benchmarks provide a stronger signal than surface similarity for code generation quality [11, 12], but even passing tests only verify claimed behaviour and leave latent defects untouched [13]. A survey of LLMs for code generation [14] lists the integration of formal verification tools within the LLM pipeline as an open safety direction, rather than a solved problem. Empirical studies of code generation failure modes [15] characterise which failures occur most often and how self–critique, multi–model collaboration or retrieval–augmented generation can enhance performance; however, all of these frameworks stop at execution–based failure rather than at reasoning–code alignment.

Faithfulness research targets reasoning, not code. On the reasoning side, Chain–of–Thought (CoT) prompting is known to elicit intermediate steps [16], but these steps do not have to reflect the model's underlying computation, and recent work has shown that language models do not always say what they think [17]. Follow–up work has formalised CoT faithfulness [18, 19] and inspected reasoning chains at a finer granularity through step–level verifier training and process supervision [20]. Typed Chain–of–Thought [21] synthesises reasoning steps into typed trees that can be validated, and CRV [22] builds an attribution graph of model computation and infers whether the reasoning is correct from structural fingerprints. These approaches address the correctness of reasoning rather than whether the emitted AutoML code implements the reasoning.

The intersection of these three gaps leaves a clear opening: an AutoML pipeline that passes every available check (compiles, runs, hits the metric, has a plausible–looking plan) can still be operationally misleading, and no existing framework is designed to detect this specific failure mode. This project aims to close that gap.

1.4 Objectives

The objectives of the project are:

1. To define reasoning–code alignment as a verification question in its own right, kept separate from whether the plan is a good strategy, whether the code meets a code specification, whether the code runs, and whether it reaches a good score.

2. To build a working, reproducible pipeline that generates AutoML code with an LLM under three prompting conditions (naive, schema–aware and meta–feature–guided), runs it in a bounded subprocess, and records the plan, the code, the execution outcome and the predictive score for every run.

3. To implement a first, deliberately limited faithfulness check that, for each meta–feature–guided run, tests two things mechanically: whether every action the plan names actually appears in the generated code, and whether every dataset statistic the plan cites matches the real meta–features within a small tolerance.

4. To evaluate meta–feature–guided prompting on a controlled 270–cell sweep (6 datasets $\times$ 3 conditions $\times$ 3 LLM backends $\times$ 5 seeds), measuring runnability, iteration count, error mix, predictive score and faithfulness, and to run the faithfulness check on the real meta–feature runs.

5. To state honestly what this first check cannot yet do, and to describe — as a proposed design and future work, not a delivered system — the fuller claim–by–claim verifier that would be needed to close the gap.

1.5 Scope

The project targets *compositional Python tabular classification and regression* pipelines built with scikit–learn [23] and its `Pipeline`, `ColumnTransformer`, estimator, scoring and custom–transformer abstractions. Within this setting, the *implemented* faithfulness check tests two things for each run: whether each operator or estimator named in the plan (for example a scaler, an imputer or a specific classifier) appears in the generated code, and whether each dataset statistic the plan cites (for example a missing–value ratio or a class–balance figure) matches the value actually computed from the data. A *fuller* verifier — designed in Chapter 3 but not implemented in this project — would additionally check:

- the columns to which each operator is applied,
- the parameters and ordering of the pipeline,
- leakage–relevant properties such as whether a scaler is fitted before or after the train/validation split,

- evaluation metrics and class–balance constraints.

These richer checks are stated as a target, not as a delivered capability.

The following items are explicitly *out of scope* for the first version of the system:

- Deep–learning pipelines and distributed training systems; multi–modal workflows; notebook pipelines with hidden state; AutoML frameworks other than scikit–learn–style tabular pipelines.
- Verification of the plan itself (whether the chosen strategy is a good strategy for the task) — the verifier only checks implementation faithfulness relative to a given plan, not the plan's merit.
- Mechanistic faithfulness in the sense of CRV [22]: the project checks *artifact–to –artifact* conformance (plan → emitted code), not conformance between the model's internal computation and its emitted reasoning.
- Automatic repair of misaligned pipelines; the verifier is designed to report, not to patch.

1.6 Contributions

The contributions of the project are of three kinds — conceptual, engineering and empirical.

1. **(Conceptual.)** A clear separation of reasoning–code alignment from reasoning correctness, program correctness, execution success and predictive performance, stated as the fourth question Q_A above.
2. **(Conceptual, proposed.)** A design (Chapter 3) for a fuller verifier: a structured form of the plan and of the code so that each plan claim can be matched to the part of the code that realises it, together with a mismatch taxonomy for AutoML pipelines. This design is specified, not implemented, and is not evaluated in this report.
3. **(Engineering.)** A running, reproducible pipeline — five FastAPI backend services (data, meta–features, generation, analysis and gateway) and a Streamlit front–end, a Redis–backed job queue and worker, a configurable LLM runner (Ollama cloud models in the reported runs), a bounded subprocess executor and a PostgreSQL database with local–filesystem artefact storage — that records the plan, code, execution trace and score of every run.

4. **(Engineering.)** A first mechanical faithfulness check on the project's `main` branch that, for every meta–feature–guided run, tests each cited action against the generated code and each cited statistic against the real meta–features, and returns a single pass/fail verdict for the run.

5. **(Empirical.)** A controlled 270–cell study of meta–feature–guided prompting, measured along five axes (runnability, iteration cost, error mix, predictive score and faithfulness), providing evidence towards the research questions RQ1–RQ5 (defined in Chapter 2). Running the check on the real meta–feature runs flagged seven pipelines that executed and scored well but whose stated reasoning did not match their code.

Implementation and validation status. To keep the claims of this report honest, the work is separated into three levels throughout. *Designed* items — the fuller claim–by–claim verifier, the mismatch taxonomy and the baseline comparison protocol — are specified in Chapters 3 and 4 but are not implemented in this project. *Implemented* items — the five backend services and the Streamlit front–end, the three prompt conditions, the bounded subprocess executor and the first faithfulness check — exist and run on the project's `main` branch. *Validated* items — the prompting study and the first faithfulness check — are supported by the 270–cell sweep in Chapter 6. The report never presents a designed item as though it were implemented or validated, and every number in Chapter 6 is traceable to the recorded experimental run.

1.7 Organization of the Report

The remainder of this report is organised as follows.

Chapter 2, Literature Review, surveys LLM–based AutoML, code and reasoning verification, formal typed and graph–based verification, and identifies the research gap that motivates the project.

Chapter 3, Proposed Methodology, defines the four correctness questions and the reasoning–code alignment relation, and sets out the *proposed* design for a fuller verifier — structured forms of the plan and the code, the matching rules, the mismatch taxonomy and the associated measures — which is specified but not implemented in this project.

Chapter 4, System Design, presents the end–to–end architecture, the execution and evaluation pipeline, the module decomposition, the data–flow and the storage schema.

Chapter 5, Implementation, describes the development environment,

the implementation of each module, the integration with the Ollama LLM runner, the bounded subprocess execution layer and the storage of faithfulness verdicts.

Chapter 6, Results and Discussion, reports the experimental protocol and the 270–cell sweep, its results across the five research questions (runnability, iteration cost, error mix, predictive score and faithfulness), the statistical analysis, and a discussion of what the faithfulness check did and did not catch.

Chapter 7, Conclusion and Future Scope, summarises the work, states the limitations and threats to validity, and outlines realistic extensions of the framework.

The appendix presents supporting material and the mapping of the project to Course Outcomes, Program Outcomes and Programme Specific Outcomes.

Chapter 2

Literature Review

2.1 Introduction

This chapter surveys the research that this project builds upon and locates the project in the current literature. The survey is organised into three parts: LLM–based AutoML systems; verification of generated code and of reasoning; and formal, typed and graph–based verification of reasoning. The parts are treated separately because they correspond to different correctness questions (as formalised in Chapter 3) and because each has developed its own evaluation style. The chapter closes with a synthesis of the research gap that the proposed framework addresses.

2.2 Existing Systems

2.2.1 LLM–Based AutoML Pipelines

The simplest shape of an LLM–driven AutoML system is a feature engineering agent that takes a tabular dataset and emits candidate feature constructions. CAAFE [3] is a representative instance in which the LLM is used to propose context–aware feature engineering steps and to iterate on them using data–driven feedback. LLMFE [24] extends this idea into an evolutionary loop in which feature engineering is framed as program search guided by execution feedback.

Data–centric AutoML is served by CatDB [25], which uses data–catalogue–guided prompts to construct ML pipelines, and by FlowPipe [26], which synthesises data–preparation pipelines using conditional generative flow networks and LLM–derived priors over multi–channel preprocessing. Agentic systems such as DS–Agent [5] and SELA [6] go further by orchestrating planning, retrieval and search over candidate pipelines, with case–based reasoning or tree–search guiding the exploration.

Full–pipeline systems such as AutoML–Agent [7] include a multi–agent architecture that spans planning, code generation and verification. It performs three verification steps: *request verification* (ReqVer), which assesses whether the user's instructions are clear and adequate for the ML task; *execution verification* (ExecVer), which checks — by prompting, before any code is run — whether candidate pipelines satisfy the user's requirements; and *implementation verification* (ImpVer), which validates the outcomes of code that has been compiled and ex-

ecuted. In this way AutoML–Agent explicitly raises the question of verification inside LLM–driven AutoML, but these three steps do not establish a one–to–one relationship between individual plan assertions and program implementations. A successful implementation–verification verdict therefore does not entail plan–program parity. The broader survey of LLMs for building and optimising ML workflows [8] catalogues this landscape of systems and the open problems that follow from it.

2.2.2 Code and Reasoning Verification

Program verification has a long history. Classical foundations include Hoare's axiomatic basis for computer programming [27], abstract interpretation [28] and model checking [29]. These techniques connect programs to specifications through rules based on assertions, abstract interpretation, model checking or runtime evidence. They do not, by themselves, consume LLM reasoning: an LLM plan has to be lowered into a restricted specification language first, after which the usual verification machinery applies on representable claims rather than on universally correct machine learning behaviour.

Executable code benchmarks have shown that behavioural tests give a stronger signal than surface similarity [11, 12]. However, tests only demonstrate claimed behaviour, and code generation can leave latent bugs intact [13]. A survey of LLMs for code generation [14] catalogues execution–based benchmarks and lists the integration of formal verification tools within the LLM pipeline as an open safety direction, rather than as an existing capability. Empirical studies of code–generation failure modes [15] offer surface and runtime error taxonomies alongside decision frameworks that directly relate failure patterns to actionable strategies in enhancing performance, including through self–critique, multi–model collaboration or retrieval–augmented generation. Execution–based failure, however, is exactly where these frameworks stop; the project in this report takes over from that point and treats reasoning–code alignment as the next verification layer.

2.2.3 Formal, Typed and Graph–Based Verification of Reasoning

Chain–of–Thought (CoT) prompting elicits intermediate reasoning steps [16]. These steps are not required to reflect the model's underlying computation; recent work has shown that LLMs do not always say what they think [17] and has extended CoT–faithfulness evaluation from proprietary models to open–weight reasoning models [18], providing a qualitative basis for the faithfulness problem.

Step–level verifier benchmarks and process supervision [19, 20] inspect chains at a finer granularity than final answers. These approaches address the problem of inspectable reasoning mechanisms, which is known to be difficult for LLMs to resolve on their own [30].

Typed Chain–of–Thought [21] synthesises reasoning steps into typed trees that can be validated, demonstrating that the tractability of prose can be preserved under constrained representations, although the resulting certificate is about the reasoning trace rather than the generated AutoML source code. CRV [22] builds an attribution graph of model computation and predicts the correctness of reasoning based on the graph structure. In both cases, the verified object is an internal or near–internal representation of the model, not the executable pipeline that an AutoML plan is lowered to.

Structured chain–of–thought has been shown to help code generation [31], but structured generation alone is not a post–generation proof that all stated steps appear in the code exactly as stated. Critique and self–refinement are responses to plan errors or evaluator feedback [9, 10]; a different but executable pipeline can be accepted whenever the oracle does not return one.

2.3 Review of Research Papers

Table 2.1 summarises how representative recent work compares with the proposed framework along five axes: the domain addressed, the reasoning artifact consumed, the kind of program artifact produced, whether the system actually executes code, whether it targets AutoML, and whether it establishes a mechanical plan–code link.

Table 2.1: Representative approaches relative to the proposed verification layer.

Work	Domain	Reasoning artifact	Program artifact	Exec.	AutoML
AutoML–Agent [7]	Full–pipeline AutoML	Plan guides agents	Implementation run and evaluated	✓	✓
CatDB [25]	Data–centric ML	Catalogue–guided prompts	Generated pipeline executed	✓	✓
FlowPipe [26]	Data–prep pipelines	LLM–derived priors	Policy–generated pipeline	✓	✓
Typed CoT [21]	Formalised reasoning	Typed proof/dataflow	Not AutoML code	–	–
CRV [22]	Mechanistic verification	Latent attribution graph	No pipeline code	–	–
Failure–Aware [15]	Generic code generation	Absent	Program + error taxonomy	✓	–
Test–based evaluation [11, 12]	Code generation	Usually absent	Program against tests	✓	–
Proposed framework	LLM–driven AutoML	Atomic typed claims	AST (pipeline graph proposed)	Complementary	✓

Three patterns emerge. First, systems that execute code either do not consume plans at all (code benchmarks) or consume them only as loose context (full–pipeline AutoML agents). Second, systems that formalise reasoning (typed CoT, CRV) operate on representations that are not the AutoML program and therefore cannot certify what the emitted pipeline does. Third, the verification layers that do exist inside LLM–AutoML systems (for example the three AutoML–Agent verifiers) check instruction well–formedness, request satisfiability and runtime success, but not a per–claim correspondence between the plan and the generated code.

2.4 Research Gap

The synthesis above yields a precise gap: *LLM–driven AutoML systems can check whether a request is actionable, whether code runs, and whether outcomes meet thresholds; reasoning–verification methods can check properties of the rationale; but no reviewed method provides a code–linked mechanical step that establishes whether the emitted AutoML code implements the stated plan.*

This matters because, in multi–agent systems, plans are the artifacts that

are presented to users, that record intended experimental design, that gate human approval and that direct delegation. A manager can approve one plan while a coding agent realises another. If the code diverges, the explanation offered to the user is operationally misleading even when performance is high, and downstream auditing can be based on a document that no longer describes the deployed system.

Accordingly, the project in this report sets five research questions that follow directly from the gap identified above:

- **RQ1:** Can an AutoML plan be transformed into a representation mechanically comparable with an executable pipeline?
- **RQ2:** Can alignment verification detect discrepancies missed by execution–only and performance–only checks?
- **RQ3:** Which categories of reasoning–code misalignment arise in practice?
- **RQ4:** How does alignment vary with the LLM, the reasoning format, the task and the pipeline complexity?
- **RQ5:** Can mismatches be localised before final evaluation or deployment?

The associated hypotheses are: H_1, execution success does not imply alignment; H_2, mechanical alignment discriminates aligned from misaligned pipelines more reliably than execution success; H_3, mismatches form stable AutoML–specific categories; and H_4, mismatch probability increases with operation count and dependency depth. These are stated as hypotheses rather than settled findings. Chapter 6 reports preliminary evidence from the 270–cell study: H_1 is supported, since seven pipelines executed and scored well yet failed the faithfulness check. H_2 is illustrated by those same detections but is not quantitatively tested here — the head–to–head comparison against execution– and performance–only baselines is left to future work, together with the fuller protocol needed to evaluate H_3 and H_4 at scale.

2.5 Summary

Existing LLM–AutoML systems demonstrate that end–to–end pipelines can be built by language models, but their verification layers focus on request validity, runtime success and headline performance. Verification of reasoning in the LLM literature focuses on faithful or structured chains and attribution graphs, but these are not the AutoML pipeline that is deployed. Classical program verification offers the formal apparatus for specification–based reasoning, but requires a

specification — which an LLM plan does not readily provide. The combination of these observations motivates a dedicated *reasoning–code alignment* verification layer for LLM–driven AutoML, which is the subject of the next chapter.

Chapter 3

Proposed Methodology

3.1 Overview

The proposed solution is a dedicated verification layer that sits between an LLM–driven AutoML planner and the executable Python pipeline that the coding agent emits. Instead of judging the pipeline only on whether it compiles, runs or reaches a target metric, the layer answers a strictly stronger question: does the emitted code mechanically implement the plan it was prepared from?

The methodology is organised around five stages:

1. A **task specification** that captures everything a plan is allowed to assume about the problem.
2. A **Reasoning Intermediate Representation (RIR)** that lowers the plan into atomic typed claims.
3. A **Program Intermediate Representation (PIR)** that lowers the generated code into a directed attributed multigraph covering AST, dataflow and library semantics.
4. A **semantic extraction and mapping** stage that normalises operator calls into a bounded predicate language and constructs claim–to–witness mappings.
5. A **verdict and metrics** stage that assigns three–valued verdicts per claim and reports coverage, weighted alignment, critical mismatch and localisation measures.

The remainder of this chapter describes each stage, specifies the alignment relation and metrics formally, presents the mismatch taxonomy, and lists the baselines against which the verifier is compared.

> **Scope of this chapter.** This chapter specifies the *proposed* framework in full. It is a design, not a report of delivered software. What is actually built and evaluated in this project is a single, deliberately narrow check on the `main` branch (Chapter 5): for each meta–feature–guided run it tests operation *presence* (whether each action the trace

names appears in the generated code) and meta–feature *value grounding* (whether each cited statistic matches the real meta–features within a tolerance), and returns a single boolean faithfulness verdict. Everything else described below — the graph–based (multigraph) Program IR, the set–valued mapper, the three– and four–valued verdicts, the full mismatch taxonomy of Table 3.1, the evaluation metrics and the baseline comparison — is proposed and is *not* implemented or evaluated in this project. The reader should treat every mechanism below as proposed unless Chapter 5 marks it implemented.

3.2 System Requirements

3.2.1 Hardware Requirements

The verifier itself is CPU–only and is comfortable on commodity developer hardware. The LLM backends used to generate plans and code are the main hardware driver.

- A 64–bit workstation or server with at least 16 GB of RAM and an SSD for pipeline execution and corpus storage.
- An LLM backend. The reported experiments used hosted `Ollama` cloud models (for example a 120B model), so no local GPU was required; running a model locally instead needs a CUDA–capable GPU with enough memory for that model.
- Optional access to a Redis–capable host for job orchestration when the pipeline is deployed as a service.

3.2.2 Software Requirements

- Python 3.10 or later, with `scikit-learn` [23], `pandas`, `numpy` and `pydantic`. The implemented verifier uses only the standard–library `ast` module; the proposed graph–based Program IR additionally requires `networkx` and `libcst`, which are not yet a project dependency.
- `FastAPI` (with `uvicorn`) and `rq`/`Redis` for the HTTP services, job queueing and run–worker.
- An LLM runner with a configurable backend (the reported runs used hosted `Ollama` cloud models) so that the generation model can be swapped without changing the rest of the pipeline.

- A bounded execution environment for running generated code (a separate Python subprocess with a wall–clock timeout). Container–level isolation with resource and filesystem limits is proposed, not implemented.
- LaTeX, for reproducing this report and generated figures.

3.3 Proposed Approach

The verifier consumes four objects: a *task specification* T, a *reasoning artifact* R, a *code artifact* C and an *execution trace* E. It produces a per–claim verdict table, a mismatch summary and a set of aggregate metrics.

3.3.1 Task Specification

The input records the dataset reference, the target column, the task type (binary / multiclass / regression), the allowed or prohibited models, the preprocessing constraints, the evaluation metric, the split policy, the resource limits and the required outputs. Each field receives a provenance tag — `USER`, `INFERRED` or `DEFAULT` — so that an unstated agent assumption is never silently treated as a user requirement.

3.3.2 Reasoning Intermediate Representation (RIR)

Unrestricted prose cannot be used as a mechanical specification. The planner (or an extraction step applied to free–form reasoning) emits a readable rationale together with a structured RIR. Each atomic claim r_i has the form

$$r_i = \langle id, o, s, a, \theta, d, q, w, \pi \rangle,$$

where o is an operation class, s is a scope, a is an operator family, θ are parameters, d is an ordering or dependency predicate, q is a qualifier, w is a criticality and π is provenance. For example, a claim can be written as

```
R2: TRANSFORM(scope=numeric,
              family=standardization,
              fit=train_only,
              before=model, weight=2).
```

Claims cover loading, splitting, imputation, encoding, scaling, feature selection, dimensionality reduction, model choice, fitting, prediction, evaluation and constraints. Meaningful but non–operational claims are retained as `NONEXECUTABLE` and are not counted as aligned.

Free–form plans require a separate extraction step. To avoid circular evaluation, extraction errors are measured against human–annotated claims; direct structured emission is the primary condition and free–form extraction is a harder comparison.

What the reasoning trace actually is (and its limitation). The design above treats the reasoning as a plan produced *before* the code. In the system actually built and evaluated here, the reasoning trace is not captured independently of the code. Under the meta–feature–guided (B2) condition the LLM is asked to return a single, self–contained Python script that, when run, prints a `SCORE:` line and then a `REASONING:` line carrying a small JSON object of decisions (the step, the cited meta–feature, the `observed_value` and the `action`). The faithfulness check reads that JSON from the script's output and tests it against the same script's code and the real meta–features. The trace is therefore *post hoc*: the code and its stated reasoning come from one generation step, so the check measures the internal consistency of a single LLM response — whether the script cites an action it did not import, or a number it did not read correctly — rather than the divergence of code from a plan fixed in advance by a human or by a separate planning stage. Capturing an independent pre–code plan would require a distinct planning step and is left to future work.

3.3.3 Code Generation and Program IR

The coding agent receives the task specification and the plan, and emits source code without verifier labels. Source, environment, seed and transcript are versioned, and failed programs are preserved as outcomes.

Python is parsed into an AST and lowered into a directed attributed multigraph

$$\mathcal{P} = (V_P, E_D, E_C, \lambda, \eta),$$

where V_P contains data objects, transforms, estimators, splits, metrics and predicates; E_D records dataflow; E_C records control or calls; λ maps nodes to normalised semantics; and η stores arguments, locations and types.

An AST identifies syntax but inadequately represents pipeline order. A runtime trace observes only one path. The combined representation uses static analysis for coverage and optional instrumentation to resolve dynamic dispatch, fitted columns and values. Library adapters normalise `Pipeline`, `ColumnTransformer`, estimators, scoring strings and custom transformers.

3.4 Algorithms / Models

3.4.1 Semantic Extraction, Mapping and Verdicts

Calls are normalised into predicates such as

```
IMPUTE(num, strategy=median),
SCALE(num, semantics=zscore, fit=train),
MODEL(family=logistic, balanced=true),
EVALUATE(metric=f1, average=macro).
```

Bodies or trusted summaries resolve aliases and wrappers. Constant propagation is conservative; unknown values are left symbolic. Dataflow provenance enables leakage checks — for example, whether a scaler was fitted before the train/validation split.

For each claim, the mapper constructs program witnesses from the operation type, scope, dataflow neighbourhood and semantic family. The rules then check presence, operator semantics, parameters, order, dependency, target and metric constraints. Each report records the claim, the expected predicate, the observed evidence, the source span, the verdict, the category and the severity.

Verdicts are reported at two levels. Each individual claim receives a *three–valued* verdict — ALIGNED, MISALIGNED or UNVERIFIABLE — the last when there is not enough evidence to decide it. The claim verdicts are then aggregated, with criticality gating, into a *four–valued* pipeline verdict:

- ALIGNED — all decided claims align and no contradiction is present.
- PARTIALLY ALIGNED — only non–critical claims mismatch or remain unaccounted for.
- MISALIGNED — a critical claim fails or a contradiction is present.
- UNVERIFIABLE — there is not enough evidence to decide the pipeline either way.

Abstinence (UNVERIFIABLE) prevents uncertainty from turning into false certainty.

3.4.2 Alignment Relation

Let $\mathcal{R} = \{r_1, \ldots, r_n\}$ be the set of plan claims and $V_P = \{p_1, \ldots, p_m\}$ the set of program nodes. The mapper is set–valued:

$$M : \mathcal{R} \to 2^{V_P} \cup \{\emptyset\}.$$

A claim can require several nodes; leakage–safe scaling, for example, requires a split, a scaler fitted on the training branch and reuse on validation data.

The per–claim relation is three–valued,

$$A(r_i, M(r_i)) \in \{1, 0, \perp\},$$

where 1 denotes agreement, 0 an established mismatch and $\perp$ insufficient evidence. It combines the applicable predicates,

$$A_i = A_i^{\text{pres}} \wedge A_i^{\text{sem}} \wedge A_i^{\text{param}} \wedge A_i^{\text{ord}} \wedge A_i^{\text{dep}}.$$

A three–valued conjunction returns 0 if any required predicate is false, 1 if all are true, and $\perp$ otherwise. In the implemented `main`–branch check only the first two conjuncts are evaluated — presence (A^{pres}) and meta–feature value grounding (a form of A^{sem}) — and they are reduced to a single boolean per run; the parameter, ordering and dependency conjuncts belong to the proposed design.

3.5 Evaluation Metrics

These metrics are part of the proposed design. None of them is computed in this project's experiment; Chapter 6 reports success rate, predictive score, error categories and the boolean faithfulness verdict, not the quantities below. They are defined here so that a fuller verifier, if built, would have a precise and non–inflatable reporting target.

Let $v_i = 1$ if claim i is decided, $a_i = 1$ only for a decided match, and $w_i > 0$ be a criticality weight assigned per claim type (for example, a leakage–relevant ordering claim carries more weight than an optional feature–selection claim).

- **Claim coverage** — the fraction of plan claims the verifier was able to decide, rather than leave unverifiable:

$$C_{\text{claim}} = \frac{1}{n} \sum_{i=1}^{n} v_i.$$

- **Weighted semantic alignment** — among the decided claims, the share that matched, with more important claims (for example a leakage–relevant ordering claim) weighted more heavily:

$$S_{\text{align}} = \frac{\sum_i w_i v_i a_i}{\sum_i w_i v_i}$$

when the denominator is non–zero. Coverage must be reported alongside

the score.

- **Critical mismatch rate** — the fraction of safety–critical claims (the set K) that were contradicted:

$$\mathrm{CMR} = \frac{\sum_{i \in K} \mathbf{1}[A_i = 0]}{|K|}.$$

- **Localisation** — how well predicted mismatch categories and their source locations match human labels, measured by category macro–F1 and source–span overlap. False–alignment and false–misalignment rates are reported separately.

Coverage is reported alongside the alignment score so that a high value cannot be achieved by labelling most claims UNVERIFIABLE and declaring the handful of remaining claims aligned.

3.5.1 Semantic Equivalence

Alignment cannot be decided by string equality. The design uses four levels of evidence: *lexical* (which symbols are candidate matches), *structural* (AST and graph patterns, or an operator's role in the program), *semantic* (the operator's meaning, taken from a trusted summary or an equivalent rewrite), and *behavioural* (agreement on a bounded set of probe inputs over a declared input domain). Lexical evidence alone cannot certify a claim, and behavioural agreement over probes does not prove universal semantic equivalence. Without a trusted contract, a custom function that relies on reflection, external state or undecidable dispatch is labelled $\perp$ (unverifiable).

For instance, `StandardScaler` and a custom $(x - \mu)/\sigma$ may match if such code fits μ and σ on training data, uses the same variance and zero–variance conventions, and transforms only the intended columns. Addition and subtraction observed on their own are not enough.

3.5.2 Worked Example

To make the representations concrete, consider a single preprocessing decision on the Titanic dataset. The LLM is given the dataset meta–features and emits the natural–language reasoning:

> *"The `Fare` column is heavily skewed with many outliers (outlier ratio* $0.128 > 0.05$*), so I scale it with `RobustScaler` rather than `StandardScaler`."*

(1) RIR claim. This lifts to one atomic typed claim:

```
C1: TRANSFORM(op=scale, scope={Fare},
              family=robust_scaling,
              meta_feature=
                distributional.outlier_ratio_per_numeric.Fare,
              observed_value=0.128, threshold=0.05,
              action=RobustScaler, weight=medium).
```

(2) Generated code. The coding step emits, among other lines:

```
from sklearn.preprocessing import RobustScaler
from sklearn.compose import ColumnTransformer
pre = ColumnTransformer([("num", RobustScaler(), ["Fare"])])
```

(3) PIR. The program lowers to nodes including a call `RobustScaler()` imported from `sklearn.preprocessing`, wired into a `ColumnTransformer` whose column literal set is `{Fare}`.

(4) Mapping and verdict. The claim `C1` maps to the `RobustScaler` node. The verifier checks: the cited action `RobustScaler` is present in the code ($A^{\text{pres}} = 1$); the cited `observed_value` 0.128 equals the real meta–feature within tolerance (A^{sem} value–grounding $= 1$); and, in the full design, the applied column `Fare` and the fit–after–split ordering also hold. The verdict is ALIGNED.

(5) The misalignment case. Had the code instead used `StandardScaler` while the trace still claimed `RobustScaler`, then $A^{\text{pres}} = 0$ and the verdict is MISALIGNED — even though the pipeline compiles, runs and may reach an identical score. This is precisely the failure that execution– and performance–based checks cannot see, and it is the check that the implemented `main`–branch verifier (Chapter 5) already performs. Figure 3.1 illustrates the full mapping.

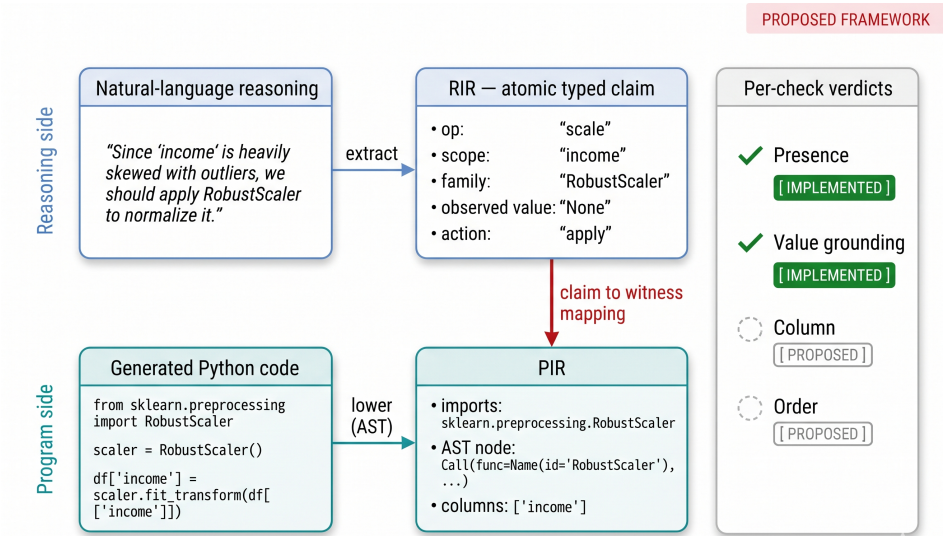


Figure 3.1: Worked RIR–PIR claim–to–code mapping for a single `RobustScaler` decision on the `Fare` column.

3.6 Mismatch Taxonomy

The proposed taxonomy of reasoning–code misalignment is given in Table 3.1. It is defined so that the mechanical condition of each category can be checked from RIR and PIR evidence, and so that each category can be weighted by severity. Secondary tags are permitted; placing a scaler before a split is both an ordering and a data–handling mismatch.

Table 3.1: Proposed taxonomy of AutoML reasoning–code misalignment.

Category	Mechanical condition	Example	Severity
Missing implementation	No reachable witness	Feature selection planned but absent	Medium–High
Contradictory implementation	Observed predicate conflicts	Preserve rows planned; code drops missing rows	High
Model mismatch	Estimator family incompatible	Logistic model planned; forest used	High
Parameter mismatch	Operator matches but value differs	Five folds stated; code uses three	Medium–High
Pipeline–order mismatch	Precedence or fit scope violated	Scaling fitted before split	High
Evaluation mismatch	Metric differs	Macro F1 stated; accuracy used	High
Data–handling mismatch	Columns, target, split or sampling differs	Stratification absent	High
Constraint mismatch	Resource or admissibility rule violated	Sparse model required; ensemble used	Medium–High
Unsupported claim	No operational definition	Use a stable model	Unverifiable
Dead reasoning	Matching node is off effective path	Selector constructed but unused	Medium
Extraneous operation	Consequential unstated step added	Target–dependent preprocessing added	Contextual

3.7 Baselines

The proposed evaluation compares five systems. This comparison is part of the proposed protocol and is *not* carried out in this project (see Chapter 6); it is described here to define what a fair assessment of the fuller verifier would involve.

1. **Execution–only**: aligned if execution succeeds.
2. **Performance–only**: aligned if execution completes and the requested performance threshold is met.
3. **LLM judge**: a separate model assesses alignment by reasoning over the plan and code without access to program analysis tools.
4. **Keyword/API matching**: claims are considered aligned when they match Python method names and metric strings.

5. **Mechanical verifier**: performs full RIR–PIR mapping with semantic normalisation, program–graph constraints, and uncertainty estimation.

Table 3.2 summarises what each baseline is capable of doing.

Table 3.2: Baseline capabilities.

Method	Runs	Plan	Graph	Localizes
Execution–only	✓	–	–	–
Performance–only	✓	Part.	–	–
LLM judge	Opt.	✓	–	Text
Keyword match	–	✓	–	Token
Mechanical verifier	Opt.	✓	✓	Claim

3.8 Data Collection and Preprocessing

Two corpora are needed.

Natural corpus. Unaltered LLM plans and code. Claims are extracted and program evidence is mapped by two annotators independently; a third has the final say when disagreements occur. Each annotator estimates alignment, category, severity and source span. Reported numbers include (1) agreement per claim extraction, (2) ternary labels and (3) taxonomy classes.

Seeded–mutation corpus. Well–aligned pipelines are the basis. A controlled mutation deletes an operator, swaps a model or a metric, changes a parameter, factors out transforms, alters the split policy, disconnects one step from another and violates any constraints. Mutations maintain syntax, and some subset must guarantee successful execution. This is a direct test of the motivating case. Negative controls consist of equivalent refactorings and aliases, wrappers, and semantically equivalent implementations. Mutation templates must retain names and comments (if possible) to prevent keyword baselines from taking advantage of simple cues; the point is that when code is mutated, only a semantic change is introduced where humans intended, and nothing else.

3.9 Summary

The proposed methodology formalises reasoning–code alignment as a set–valued mapping from atomic typed claims (RIR) to program witnesses in a directed attributed multigraph (PIR), with a three–valued verdict space that includes a dedicated UNVERIFIABLE escape. A bounded predicate language normalises scikit–learn calls, an AutoML–specific taxonomy describes mismatch categories, and a

proposed metric suite (coverage, weighted alignment, critical mismatch and localisation) would summarise the result in a way that cannot be inflated by abstention. A set of baselines, including an LLM–judge, is specified so that the added value of the mechanical check could be measured rather than assumed. Of all this, only the presence and value–grounding check is implemented and evaluated in this project; the next chapter presents the system architecture within which that check runs on `main`, while the fuller design remains proposed.

Chapter 4

System Design

4.1 System Architecture

The system is organised as three logical tiers — an *ingestion and generation* tier, a *run and verification* tier, and an *analysis* tier — that share a persistent store and a job queue. The tiers correspond respectively to turning a user request into a plan and runnable code, to executing that code under supervision and verifying its alignment with the plan, and to answering the five research questions over the accumulated run records.

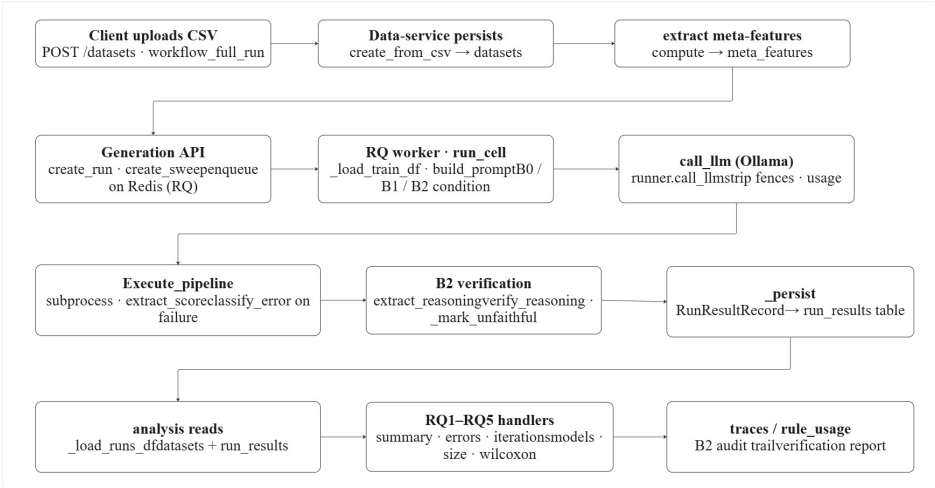
4.1.1 Ingestion and Generation Tier

The ingestion and generation tier exposes HTTP interfaces backed by **FastAPI** services (served by `uvicorn`). A client uploads a CSV via `POST /datasets` on the data service and triggers a full workflow via the gateway. The data service persists the dataset into the `datasets` table; the meta–features service computes and caches the dataset's meta–feature profile into the `meta_features` table, keyed one–to–one by dataset. The generation service then records a `sweep_jobs` row and enqueues one job per cell on the **Redis Queue (RQ)**. Note that "RQ" here denotes the Redis Queue job runner; the research questions of this project are written **RQ1**–**RQ5** and are a separate concept.

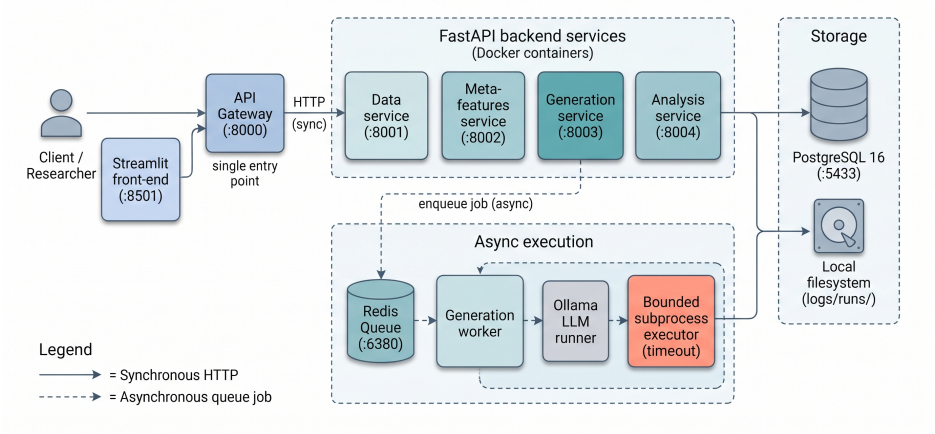
4.1.2 Run and Verification Tier

The run and verification tier is driven by a **Redis Queue (RQ) worker** that executes the `run_cell` task. The worker loads the training dataframe, builds the generation prompt for one of the three prompt conditions (`B0`, `B1`, `B2`) and calls the configurable LLM runner (reached through `call_llm_with_usage`; the reported runs used hosted Ollama cloud models), recording token usage.

The emitted pipeline is executed inside a bounded Python subprocess (a separate process under a wall–clock timeout — not container-level isolation) that captures stdout and stderr, extracts the achieved score, classifies failures into one of thirteen error categories and preserves failed outcomes. For `B2` runs a verification stage follows: the **implemented verifier** parses the reasoning trace and, for each cited decision, checks that the named action appears in the generated code's AST and that the cited statistic matches the real meta–feature, yielding a



(a) Runtime flow of a single run: upload and meta–feature extraction, sweep enqueue, the `run_cell` worker (prompt build, `call_llm`, bounded–subprocess execution, B2 verification), persistence to `run_results`, and the read–only RQ1–RQ5 analysis.



(b) Component and deployment view: the Streamlit front–end and clients reach the five FastAPI services through the gateway (synchronous HTTP); the generation service enqueues jobs on Redis for the worker, which drives the Ollama runner and the bounded subprocess executor (asynchronous); PostgreSQL and the local filesystem hold the results and artefacts. MinIO object storage is integrated in the code but was disabled for the reported runs.

Figure 4.1: System architecture: (a) the runtime flow of one run, and (b) the deployed components and their connections.

boolean `faithful` verdict that is persisted. The richer three–valued verdict with mismatch category and source span specified in Chapter 3 is the proposed design; it is not produced by the current tier (see Chapter 5).

4.1.3 Analysis Tier

The analysis tier persists one `RunResultRecord` per cell into the `run_results` table. A set of RQ1–RQ5 handlers reads `load_runs`, `df_datasets` and `run_results` and produces summaries, error counts, iteration models, size versus wall–clock curves and the verification report that answers the research questions. B2 audit trails are retained so that any aggregate claim can be traced back to the individual verifier decisions that produced it.

4.2 System Workflow

A single workflow proceeds along the three rows of Fig. 4.1a as follows:

1. **Request.** The client calls `POST /datasets` with a CSV and `POST /workflow_full_run` with the task specification. The data service persists the file and enqueues meta–feature extraction.

2. **Sweep creation.** The generation service records a `sweep_jobs` row and enqueues one job per cell on the Redis Queue.

3. **Reasoning + code generation.** The RQ (Redis Queue) worker loads the training frame, selects the prompt condition (B0/B1/B2), calls the LLM runner and receives the generated pipeline code (and, under B2, a reasoning trace).

4. **Execution.** `Execute_pipeline` runs the code in a bounded subprocess, times out on hangs, and extracts the achieved score.

5. **Verification (B2).** The implemented verifier checks each decision in the reasoning trace for action presence and meta–feature value grounding and records a boolean faithfulness verdict; the full RIR/PIR per–claim verdicts of Chapter 3 are the proposed extension.

6. **Persistence.** `_persist` writes the run record (plan, code, trace, score, verdicts) to `run_results`.

7. **Analysis.** The RQ handlers aggregate the stored records into the summary, error, iteration, model–class, size and wildcard tables, and the trace/rule–usage report.

4.3 Use Case Diagram

The three principal use cases are:

- *Submit workflow* — a researcher uploads a dataset and a task specification, selects which research questions to exercise, and starts a sweep.
- *Audit a run* — a researcher opens a single run, inspects its code, execution trace and (for `B2`) the faithfulness verdict together with the reasons recorded for any failed check.
- *Reproduce an analysis* — a researcher regenerates the aggregate tables for RQ1–RQ5 from the stored run records, without re–running the LLM.

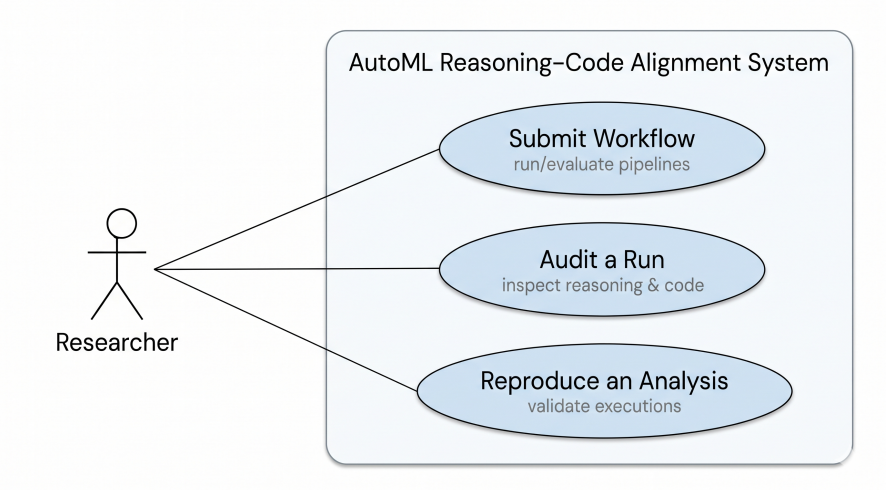


Figure 4.2: Principal use cases of the system.

4.4 Data Flow

Data flows through the system as follows:

- CSV uploads enter the data service and are indexed by a dataset identifier. Meta–features derived from the dataset are stored alongside it.
- For each cell in a sweep, the RQ worker reads the training dataframe and composes a prompt that includes the dataset reference, the task type, and the RQ–specific instructions.
- The LLM returns a single, self–contained Python script, retained verbatim. Under `B2` the script, when run, also prints a structured reasoning trace (a small JSON object of decisions) that is captured from its output. The full

PIR of Chapter 3 is proposed and is not built here; the implemented check parses the code only into the set of names it references.

- The subprocess run returns a trace object with stdout, stderr, the extracted score (or an error classification) and timing data.
- For `B2`, the verifier returns a report whose fields are the number of decisions, the number found faithful, a single boolean `faithful` flag, and a per–decision list recording the step, the cited action, whether that action is present in the code, whether the cited value matched, and human–readable reasons for any failure. The richer records with mismatch category, severity and source span belong to the proposed design. These fields join the code, trace and score into a single `RunResultRecord`.
- The analysis handlers consume `RunResultRecord`s only; no handler reaches back into the live LLM, so analyses are repeatable from storage.

4.5 Database Design

The system of record is **PostgreSQL 16**, accessed through SQLAlchemy ORM models. The schema has *four* tables, defined so that every aggregate report can be rebuilt from raw rows — the analysis layer only reads, and never writes derived values back. Figure 4.3 gives the entity–relationship overview and Figures 4.4–4.7 show the actual ORM definitions.

- **datasets** — one row per registered dataset: `id`, `name`, `source` (`openml`/`custom`), `openml_id`, `target_col`, `task_type`, `train_path`, `test_path`, `n_rows`, `n_cols`, `created_at`.
- **meta_features** — one row per dataset (enforced by a `UniqueConstraint` on `dataset_id`): the full meta–feature profile as a `JSON` `features` column, plus `computed_at`. This is the cache that the B2 prompt reads.
- **run_results** — one row per executed cell: `dataset_id`, `condition` (`B0`/`B1`/`B2`), `llm_backend`, `seed`, `success`, `test_score`, `error_category`, `iterations_used`, `runtime_seconds`, `generated_code_path`, `prompt_tokens`, `completion_tokens`, and — for B2 only — the `reasoning_trace` and `verification_report` as `JSON`.
- **sweep_jobs** — one row per enqueued sweep: `rq_job_id`, `status`, the cross–product `params` (`JSON`), and `total_cells` / `completed_cells` / `failed_cells` progress counters.

Large artefacts (generated code, long traces) are written to the local filesystem under `logs/runs/`, with a path reference kept in the row. The stack also integrates **MinIO** object storage for these artefacts, but MinIO was disabled in the evaluated deployment (its images are no longer publicly pullable), so the reported runs used the filesystem. Writes are idempotent: re–running a cell refreshes its row rather than creating duplicates.

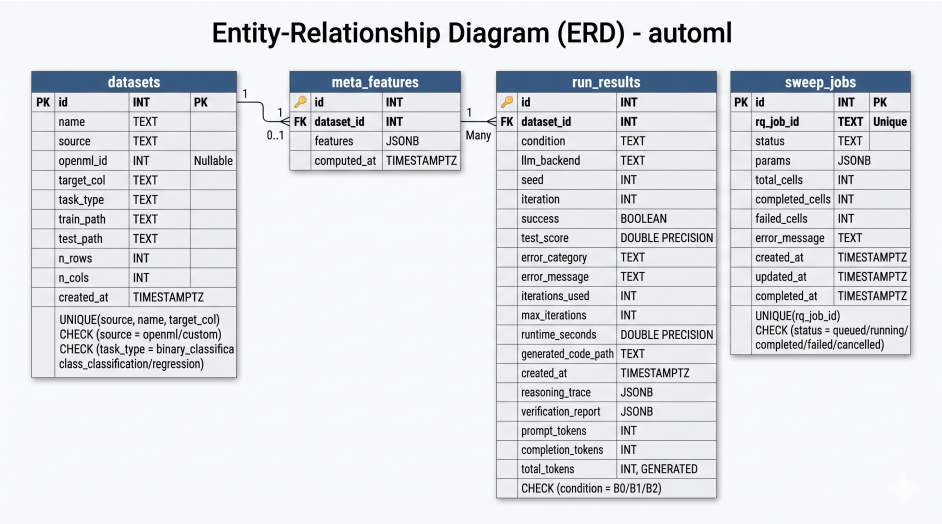


Figure 4.3: Entity–relationship diagram of the PostgreSQL schema.

```python
class DatasetRecord(Base):
    __tablename__ = "datasets"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    name: Mapped[str] = mapped_column(String, nullable=False)
    source: Mapped[str] = mapped_column(String, nullable=False)
    openml_id: Mapped[int | None] = mapped_column(Integer, nullable=True)
    target_col: Mapped[str] = mapped_column(String, nullable=False)
    task_type: Mapped[str] = mapped_column(String, nullable=False)
    train_path: Mapped[str] = mapped_column(String, nullable=False)
    test_path: Mapped[str] = mapped_column(String, nullable=False)
    n_rows: Mapped[int | None] = mapped_column(Integer, nullable=True)
    n_cols: Mapped[int | None] = mapped_column(Integer, nullable=True)
    created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True), server_default=func.now(), nullable=False
    )
```

Figure 4.4: ORM model for the `datasets` table.

```python
class MetaFeaturesRecord(Base):
    __tablename__ = "meta_features"
    __table_args__ = (UniqueConstraint("dataset_id", name="uq_meta_features_dataset_id"),)

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    dataset_id: Mapped[int] = mapped_column(Integer, nullable=False, index=True)
    features: Mapped[dict] = mapped_column(JSON, nullable=False)
    computed_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True), server_default=func.now(), nullable=False
    )
```

Figure 4.5: ORM model for the `meta_features` table; the `UniqueConstraint` enforces one profile per dataset.

```python
class RunResultRecord(Base):
    __tablename__ = "run_results"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    dataset_id: Mapped[int] = mapped_column(Integer, nullable=False, index=True)
    condition: Mapped[str] = mapped_column(String, nullable=False, index=True)
    llm_backend: Mapped[str] = mapped_column(String, nullable=False, index=True)
    seed: Mapped[int] = mapped_column(Integer, nullable=False)
    iteration: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
    success: Mapped[bool] = mapped_column(Boolean, nullable=False, index=True)
    test_score: Mapped[float | None] = mapped_column(Float, nullable=True)
    error_category: Mapped[str | None] = mapped_column(String, nullable=True)
    error_message: Mapped[str | None] = mapped_column(String, nullable=True)
    iterations_used: Mapped[int | None] = mapped_column(Integer, nullable=True)
    max_iterations: Mapped[int | None] = mapped_column(Integer, nullable=True)
    runtime_seconds: Mapped[float | None] = mapped_column(Float, nullable=True)
    generated_code_path: Mapped[str | None] = mapped_column(String, nullable=True)
    # B2 structured reasoning + mechanical audit. Both nullable — B0/B1 never
    # produce a trace, and B2 runs whose trace failed extraction/verification
    # leave these NULL and are flagged via error_category.
    reasoning_trace: Mapped[dict | None] = mapped_column(JSON, nullable=True)
    verification_report: Mapped[dict | None] = mapped_column(JSON, nullable=True)
    prompt_tokens: Mapped[int | None] = mapped_column(Integer, nullable=True)
    completion_tokens: Mapped[int | None] = mapped_column(Integer, nullable=True)
    created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True), server_default=func.now(), nullable=False
    )
```

Figure 4.6: ORM model for the `run_results` table; note the nullable B2–only `reasoning_trace` and `verification_report` JSON columns and the token counters.

```python
class SweepJobRecord(Base):
    __tablename__ = "sweep_jobs"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)
    rq_job_id: Mapped[str | None] = mapped_column(String, unique=True, nullable=True)
    status: Mapped[str] = mapped_column(String, nullable=False, index=True)
    params: Mapped[dict] = mapped_column(JSON, nullable=False)
    total_cells: Mapped[int | None] = mapped_column(Integer, nullable=True)
    completed_cells: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
    failed_cells: Mapped[int] = mapped_column(Integer, nullable=False, default=0)
    error_message: Mapped[str | None] = mapped_column(String, nullable=True)
    created_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True), server_default=func.now(), nullable=False
    )
    updated_at: Mapped[datetime] = mapped_column(
        DateTime(timezone=True), server_default=func.now(), onupdate=func.now(),
        nullable=False,
    )
    completed_at: Mapped[datetime | None] = mapped_column(
        DateTime(timezone=True), nullable=True
    )
```

Figure 4.7: ORM model for the `sweep_jobs` table.

4.6 Module Description

The system decomposes into eight modules.

1. **Data Service.** Persists uploaded CSVs, extracts and stores meta–features, and exposes `create_from_csv`.

2. **Generation API.** Orchestrates `create_run` and the per–RQ sweep creation on Redis.

3. **Prompt Builder.** Constructs the plan–emission and code–emission prompts for prompt classes `B0`, `B1`, `B2` and conditional variants.

4. **LLM Runner.** Wraps the configurable LLM backend (the reported runs used Ollama) as `runner.call_llm` with explicit input/output fences and usage accounting.

5. **Executor.** The `Execute_pipeline` bounded subprocess executor that runs the generated code, extracts the score, classifies errors on failure and times out hangs.

6. **Verifier.** The B2 module that checks, for each cited decision, action presence in the code and meta–feature value grounding, recording a boolean faithfulness verdict. Lowering the plan into the RIR and the code into the PIR to emit the per–claim verdicts of Chapter 3 is the proposed extension and is not implemented in this project.

7. **Persistence Layer.** `_persist` writes the `RunResultRecord` and keeps the audit trail required for reproducibility.

8. **Analysis Handlers.** RQ1–RQ5 handlers that render the summary, error, iteration, model, size and wildcard tables, together with the B2 audit/verification report.

Chapter 5

Implementation

This chapter reports what has actually been *built and run* on the project's `main` branch, and separates it cleanly from the parts of Chapter 3 that remain designed but unbuilt. It is the honesty anchor of the report: the empirical results of Chapter 6 are produced only by the implemented components described here.

5.1 Implementation Status

Table 5.1 classifies each component as *implemented* (exists and runs on `main`), *implemented (prototype)* (a first, deliberately narrow version), or *proposed* (specified in Chapter 3, not yet coded).

Table 5.1: Implementation status of each system component.

Component	Status
Five FastAPI services (data, meta–features, generation, analysis, gateway) plus a Streamlit front–end	Implemented
Redis Queue worker + sweep orchestration (`sweep_jobs`)	Implemented
Three prompt conditions B0 / B1 / B2	Implemented
Ollama LLM runner (three cloud backends)	Implemented
Bounded subprocess executor (separate process + wall–clock timeout) + 13–category error taxonomy	Implemented
Meta–feature extraction (four families)	Implemented
PostgreSQL persistence; generated code, traces and report sidecars on the local filesystem (`logs/runs/`)	Implemented
RQ1–RQ5 analysis endpoints, `/analysis/traces`, `/analysis/rule-usage`	Implemented
Faithfulness verifier: action–presence + value–grounding, boolean verdict	Implemented (prototype)
AST–based Program IR; existence / parameter / ordering / column checks; three–valued claim + four–valued pipeline verdicts; mismatch taxonomy; verifier CLI	Proposed
Graph–based (multigraph) Program IR; set–valued mapper; full mismatch taxonomy as emitted categories; verification dashboard	Proposed
Independent pre–code reasoning (`B2R` mode)	Proposed

5.2 Development Environment

- **Language.** Python 3.10+.
- **Services.** `FastAPI` with `uvicorn`; each service is a separate git repository with its own `Dockerfile`. The `automl-infra` repository holds the Docker Compose definition and the SQL migrations.
- **ML libraries.** `scikit-learn`, `pandas`, `numpy`, `scipy`.
- **Program analysis.** The implemented verifier uses the standard–library `ast` module only. The `networkx`/`libcst` graph layer of Chapter 3 is *not* a current dependency.

- **LLM runner.** `Ollama`, wrapped by `call_llm_with_usage` so that the backend can be swapped and token usage recorded. The evaluated backends are `gpt-oss:120b-cloud`, `gemma4:31b-cloud` and `nemotron-3-nano:30b-cloud`.
- **Storage. PostgreSQL 16** is the system of record; large artefacts (generated code, model files and trace / report sidecars) are written to the local filesystem under `logs/runs/`; **Redis 7** backs the job queue. The stack integrates **MinIO** object storage for artefacts, but it was disabled for the reported runs, as its images are no longer publicly pullable.
- **Tooling.** Per–repository `git`, `pytest` for unit tests, and this LaTeX report.

5.3 Implemented Services

The stack is five FastAPI backend services (data, meta–features, generation, analysis and the gateway) plus a Redis Queue worker and a Streamlit front–end, each with its own container (Table 5.2).

Table 5.2: Backend services, front–ends and their host ports.

Service	Port	Responsibility
`automl-data-service`	8001	Register a CSV or fetch an OpenML dataset; persist rows + metadata.
`automl-metafeatures-service`	8002	Compute / cache the meta–feature profile for a dataset.
`automl-generation-service`	8003	Enqueue code–generation jobs on Redis; a worker runs, executes and verifies them.
`automl-analysis-service`	8004	Read–only analytics: success rates, iterations, error mix, significance tests, trace and rule–usage reports.
`automl-gateway`	8000	Single entry point; composes `upload-and-extract` and `full-run` workflows.
`automl-streamlit`	8501	Streamlit front–end for browsing datasets and runs.

Shared logic lives in the `automl-reusables` library (imported by every service): the meta–feature extractors, the prompt conditions, the executor, the error taxonomy and the verifier.

5.3.1 Data Service

The data service accepts a CSV at `POST /datasets` or fetches an OpenML dataset by id, performs a deterministic stratified train/test split, writes the CSVs to a dataset–identified folder and persists a `datasets` row. It never forwards raw rows to the LLM; only derived descriptors are exposed downstream (Fig. 5.1).

```python
@app.get("/health", response_model=HealthResponse)
def health(session: Session = Depends(get_session)) -> HealthResponse:
    try:
        session.execute(text("SELECT 1"))
        return HealthResponse(status="ok", database="connected")
    except Exception:
        return HealthResponse(status="degraded", database="unreachable")


@app.post("/datasets", response_model=DatasetResponse, status_code=201)
def create_dataset_from_csv(
    file: UploadFile = File(..., description="CSV file with header row."),
    target_col: str = Form(..., description="Name of the target column."),
    task_type: Optional[TaskTypeStr] = Form(
        None, description="Auto-detected if omitted."
    ),
    seed: int = Form(42),
    test_size: float = Form(0.2, gt=0.0, lt=1.0),
    session: Session = Depends(get_session),
) -> DatasetResponse:
    ds = upload_csv(
        session, file, target_col=target_col, task_type=task_type,
        seed=seed, test_size=test_size,
    )
    return DatasetResponse.model_validate(ds)      Kinks106, 4 months ago • ch
```

Figure 5.1: Data–service ingestion path (dataset registration and split).

5.3.2 Meta-Feature Service

For each dataset the service computes one JSON meta–feature object with four families — *simple* (counts, missingness, cardinality, class balance), *distributional* (skewness, kurtosis, outlier ratio), *information* (mutual information, pairwise correlation, target entropy) and *landmarking* (decision stump, linear/NB and 1–NN reference scores) — and caches it in the `meta_features` table. Column typing is handled by small, deterministic helpers (Fig. 5.2).

```python
import pandas as pd


def is_categorical(s: pd.Series) -> bool:
    if pd.api.types.is_bool_dtype(s):
        return True
    if pd.api.types.is_integer_dtype(s.dtype) and s.nunique() < 20:
        return True
    return not pd.api.types.is_numeric_dtype(s.dtype)


def classify_columns(
    df: pd.DataFrame,
) -> tuple[list[str], list[str]]:
    numeric: list[str] = []
    categorical: list[str] = []
    for col in df.columns:
        if is_categorical(df[col]):
            categorical.append(col)
        else:
            numeric.append(col)
    return numeric, categorical


def numeric_feature_cols(df: pd.DataFrame, target_col: str) -> list[str]:
    return [
        col for col in df.columns
        if col != target_col and not is_categorical(df[col])
    ]
```

Figure 5.2: Meta–feature helpers: a column is categorical if boolean, low–cardinality integer, or non–numeric.

5.3.3 Prompt Conditions: B0, B1, B2

The three conditions are the experiment's independent variable and differ *only* in prompt content; the code that executes, scores and persists their output is identical.

- **B0 (naive).** Task description only, plus the hard requirement that every class/function be explicitly imported.
- **B1 (schema).** B0 plus a dataset–schema section: row/column counts, task type, target column, column dtypes and the first three rows.
- **B2 (meta–feature–guided).** B1 plus the full meta–feature JSON, fourteen numbered decision rules, a cite–your–reasoning instruction, and a *mandatory, machine–verified* structured reasoning trace that the generated script must print at runtime (Fig. 5.3).

```python
class B2MetaFeatureGuided(PromptCondition):
    @property
    def condition_name(self) -> str:
        return "B2"

    def build_prompt(
        self,
        task_description: str,
        meta: MetaFeatures,
        df_head: pd.DataFrame,
        target_col: str,
    ) -> str:
        dtype_lines = "\n".join(f"  - {col}: {dtype}" for col, dtype in df_head.dtypes.items())
        schema_section = (
            "## Dataset Schema\n\n"
            f"**Rows:** {meta.simple.n_rows}  |  **Columns:** {meta.simple.n_cols}\n\n"
            f"**Task type:** {meta.task_type.value}\n\n"
            f"**Target column:** {target_col}\n\n"
            f"**Column dtypes:**\n{dtype_lines}\n\n"
            f"**First 3 rows:**\n{df_head.head(3).to_markdown(index=False)}\n"
        )
        meta_section = (
            "## Dataset Meta-Feature Profile\n\n"
            f"```json\n{meta.model_dump_json(indent=2)}\n```\n"
        )
        rules_section = (
            "## Decision Rules (apply where conditions match)\n\n"
            f"{_DECISION_RULES}\n"
        )
        reasoning_section = (
            "## Structured Reasoning (mandatory — machine-verified)\n\n"
            "After the `SCORE: <number>` line, your script MUST also print exactly one line of the
```

Figure 5.3: The B2 prompt builder assembling the schema, meta–feature profile, decision rules and mandatory reasoning–trace sections.

A consequence worth stating plainly: because the B2 reasoning trace is printed by the *generated code at runtime*, it is a post–hoc self–report derived from the code, not a plan captured independently *before* the code. Capturing an independent pre–code plan (the `B2R` mode of Chapter 3) is designed but not yet wired, and is the single most important limitation of the current verifier.

5.3.4 Generation Service and RQ Worker

The generation service records a `sweep_jobs` row and enqueues one Redis Queue job per cell; a worker container pulls each job, runs `jobs.run_cell`, and retries up to `max_iterations` times, feeding the classified error back to the LLM as a hint on each retry (Fig. 5.4).

```python
def enqueue_run(
    session: Session,
    queue,  # rq.Queue or a stub with .enqueue()
    dataset_id: int,
    condition: str,
    llm_backend: str,
    seed: int,
    max_iter: int,
    timeout_seconds: int,
) -> tuple[str, dict]:
    """Enqueue a single-cell run. Returns (rq_job_id, params_snapshot)."""
    job = queue.enqueue(
        jobs.run_cell,
        dataset_id=dataset_id,
        condition_key=condition,
        llm_backend=llm_backend,
        seed=seed,
        max_iter=max_iter,
        timeout_seconds=timeout_seconds,
        job_timeout=timeout_seconds * (max_iter + 1) + 60,
    )
    return job.id, {
        "dataset_id": dataset_id,
        "condition": condition,
        "llm_backend": llm_backend,
        "seed": seed,
        "max_iter": max_iter,
        "timeout_seconds": timeout_seconds,
    }
```

Figure 5.4: Enqueuing one sweep cell as a Redis Queue job; the job runs `jobs.run_cell` on a worker.

5.3.5 Bounded Subprocess Executor and Error Taxonomy

Generated code is written to a temporary file and run in a fresh Python subprocess with a per–attempt wall–clock budget; stdout, stderr and the return code are captured and the printed `SCORE:` line is parsed. Execution failures are classified into the thirteen–category `ErrorCategory` taxonomy: a priority–ordered, `stderr`–based classifier assigns the code–failure categories (`syntax_error`, `import_error`, `api_hallucination`, `shape_mismatch`, `type_error`, `missing_name`, `timeout`, `resource_limit` and others), the leakage guard assigns `suspicious_leakage`, and the B2 verifier assigns `reasoning_unfaithful`. Infrastructure–level failures (such as a failed LLM call) are recorded under a separate `infrastructure` label, so that LLM/prompt failures are never confused with infrastructure failures (Fig. 5.5).

```python
def classify_error(stderr: str, returncode: int) -> tuple[ErrorCategory, str]:
    """Classify a failed subprocess run into an ErrorCategory and a cleaned message.

    Rules are checked in priority order — the first match wins.
    """
    message = _last_traceback_line(stderr)

    if returncode in (-9, -15):
        return ErrorCategory.TIMEOUT, message
    if "SyntaxError" in stderr:
        return ErrorCategory.SYNTAX_ERROR, message
    if "ModuleNotFoundError" in stderr or "ImportError" in stderr:
        return ErrorCategory.IMPORT_ERROR, message
    if "NameError" in stderr and "is not defined" in stderr:
        return ErrorCategory.MISSING_NAME, message
    if (
        "has no attribute" in stderr
        or "is not callable" in stderr
        or "unexpected keyword" in stderr
    ):
        return ErrorCategory.API_HALLUCINATION, message
    if (
        "shape" in stderr
        or "dimension" in stderr
        or "Could not convert" in stderr
        or "Found input variables with inconsistent" in stderr
    ):
        return ErrorCategory.SHAPE_MISMATCH, message
    if "TypeError" in stderr and (
        "categorical" in stderr
        or "string" in stderr
        or "could not convert string to float" in stderr
    ):
        return ErrorCategory.TYPE_ERROR, message
    if (
        "DeprecationWarning" in stderr
        or "FutureWarning" in stderr
        or "was deprecated" in stderr
    ):
        return ErrorCategory.DEPRECATED_API, message
    if "MemoryError" in stderr or "ResourceWarning" in stderr:
        return ErrorCategory.RESOURCE_LIMIT, message
    return ErrorCategory.RUNTIME_OTHER, message
```

Figure 5.5: The rule–based execution–error classifier (`classify_error`); the first matching rule wins.

5.3.6 Implemented Verifier (Prototype)

The implemented verifier (`execution/verification.py`) realises a deliberately narrow subset of Chapter 3, and only for B2 runs. For each decision in the reasoning trace it performs two pure–Python checks:

1. **Action presence.** Parse the generated code with `ast` into a set of referenced names (imports, called functions, attributes, bare identifiers). The decision passes if its cited `action` symbol is in that set — catching a preprocessor the LLM *claimed* but never used.

2. **Value grounding.** Follow the decision's cited `meta_feature` dotted path into the real meta–feature object and compare the LLM's cited `observed_value` within a 10^{-3} tolerance — catching a *fabricated or misremembered* number.

A run is `faithful` iff every decision passes both checks; an unfaithful run is recorded with `error_category = reasoning_unfaithful`. The current code

treats an *uncheckable* value (no numeric path to compare) as not–contradicted; the proposed three–valued verdict of Chapter 3 would instead return UNVERIFIABLE here, and closing this gap is listed as future work. This verifier is the mechanism behind the faithfulness result of Chapter 6 (Section 6.3.5).

5.3.7 Persistence and Analysis

Each cell is persisted as a `run_results` row; the B2 `reasoning_trace` and `verification_report` are stored as JSON columns and also written as `.trace.json` / `.verification.json` sidecars on the local filesystem. The analysis service exposes one read–only endpoint per research question plus `/analysis/traces` and `/analysis/rule-usage`; because it only reads persisted rows, every table in Chapter 6 can be regenerated without re–calling any LLM.

5.4 Proposed Components (Designed, Not Yet Implemented)

Beyond the `main` verifier, the parts of Chapter 3 that would make the check cover more of a plan are *not* built in this project and are not drawn upon by the 270–run study of Chapter 6. These are: the AST– and graph–based Program IR; the existence, parameter, ordering and column checks; the three–valued per–claim and four–valued pipeline verdicts; the set–valued claim–to–witness mapper; the full mismatch taxonomy as emitted categories; and a verification–result dashboard. The independent pre–code reasoning capture (`B2R` mode), which would replace the post–hoc trace with a plan fixed before the code, is likewise designed but not wired. All of these remain future work.

5.5 Algorithm: Implemented vs Proposed

The implemented `main` verifier is the compact routine of Figure 5.6 (presence and value grounding only); the flowchart of Figure 5.7 is the proposed full verifier of Chapter 3 — the RIR, the AST– and graph–based PIR, the three–valued conjunction over presence, parameters and ordering, the set–valued mapper and the aggregation — which is specified but not implemented in this project.

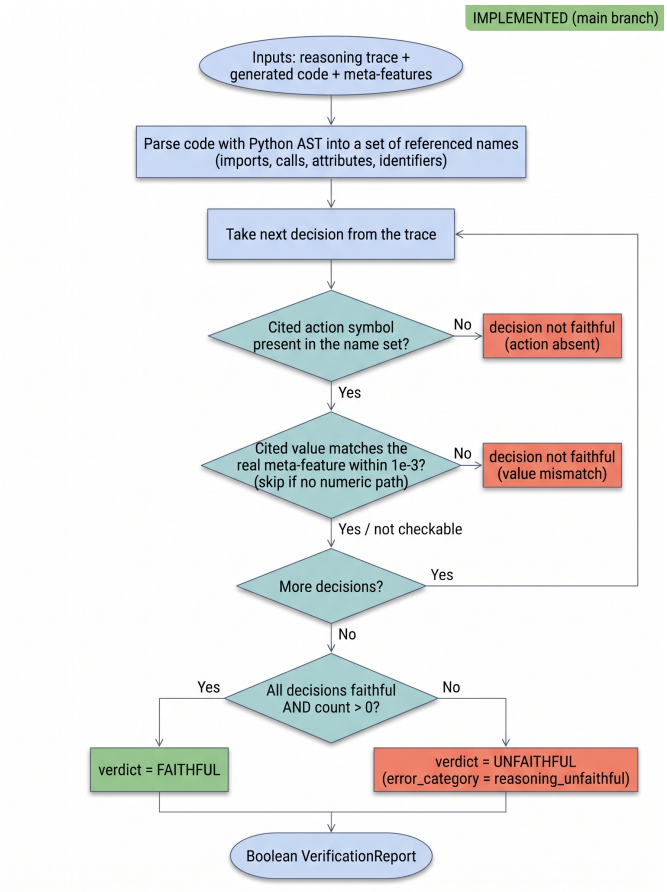


Figure 5.6: Implemented boolean faithfulness check (`verify_reasoning` on `main`): for each decision, action–presence in the AST name set and meta–feature value grounding, reduced to a single boolean verdict (an all–faithful run passes; otherwise `reasoning_unfaithful`).

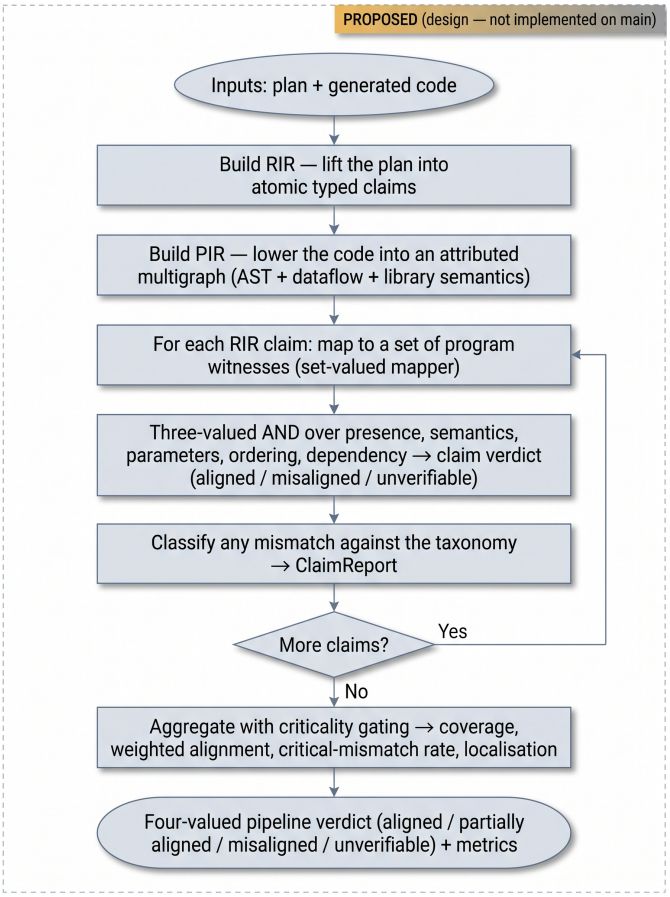


Figure 5.7: Proposed full RIR–PIR verifier (designed in Chapter 3, *not* implemented): per–claim three–valued verdicts over presence, semantics, parameters, ordering and dependency, aggregated with criticality gating into coverage, weighted alignment, critical–mismatch rate, localisation and a four–valued pipeline verdict.

5.6 Security and Privacy

The system executes LLM–generated code, so the execution environment is a security consideration. What is implemented is *bounded subprocess execution*, not a hardened sandbox: each pipeline runs as a separate Python subprocess (no shared memory with the parent) under a wall–clock timeout, with its code and output persisted under `logs/runs/` for later inspection. This bounds run time and isolates process memory, but it is an operational safeguard, *not* a security boundary — the subprocess inherits the parent environment and has the same filesystem and network access as the host user, and there is no container, OS–level, filesystem or network isolation and no memory or CPU limit. Running genuinely untrusted code safely would require adding such isolation (a container or a restricted runtime), which is future work. Two data–safety measures are in place: a leakage guard flags any near–perfect score (`>=0.995`) on a classification task for review, and raw dataset rows are never sent to the LLM runner — only meta–features and column descriptors are.

5.7 Summary

The implemented system is a running stack of five FastAPI backend services, a Streamlit front–end and a Redis Queue worker that generates scikit–learn pipelines under three prompt conditions, runs them in a bounded subprocess, classifies failures, and — for B2 — mechanically checks the reasoning trace for action presence and value grounding, persisting run records to PostgreSQL and artefacts to the local filesystem. The full graph–based verifier of Chapter 3 remains proposed. The next chapter evaluates exactly what is implemented here on a real 270–cell experimental sweep.

Chapter 6

Results and Discussion

This chapter reports measured results from a real experimental run of the implemented system (Chapter 5). Every number is traceable to the recorded run; no value is a placeholder or an estimate. Where a research question requires the full verifier of Chapter 3, which is not yet built, the question is answered as far as the implemented main–branch verifier (presence and value grounding) allows and the remainder is marked as future work.

6.1 Experimental Setup

The evaluation is a controlled **270–cell sweep**. The unit of analysis is a cell — a tuple

$$(\text{dataset}, \text{condition}, \text{LLM backend}, \text{seed}),$$

and the design is the full cross–product of **6 datasets** $\times$ **3 conditions** $\times$ **3 backends** $\times$ **5 seeds**. Each cell produces one `run_results` row.

- **Conditions.** B0 (naive), B1 (schema), B2 (meta–feature–guided), as implemented in Chapter 5.
- **Backends.** `gpt-oss:120b-cloud`, `gemma4:31b-cloud` and `nemotron-3-nano:30b-cloud`, served through Ollama.
- **Seeds.** 3001–3005 (five), fixing the train/test split and generation seed so comparisons across conditions are paired on the same cell.
- **Metric.** All six datasets are binary classification; the score is `balanced_accuracy` on a held–out test split (higher is better), chosen so that a majority–class predictor cannot score well.
- **Max iterations.** 3; a cell is a failure if no runnable, verified pipeline is produced within three attempts.

Four outcomes are tracked separately for each cell and are not conflated below: (i) *execution* — did the generated pipeline run and produce a predictive score; (ii) *predictive score* — the `balanced_accuracy` achieved when it ran; (iii) *verifier outcome* — for B2 only, whether the reasoning trace passed the faithfulness check (B0 and B1 carry no trace, so the verifier is *not applicable*); and (iv) *verified*

yield — runs that both executed and passed the faithfulness check. A B2 cell can therefore execute and produce a score yet still be an unverified outcome if the faithfulness check rejects it.

The run completed in 5 h 17 m. A leakage guard (any `balanced_accuracy` ≥ 0.995 on a classification task) fired on **zero** cells. All artefacts (generated code, traces, verification reports) are persisted, so the tables below can be regenerated from storage without re–calling any LLM.

> **Note on earlier data.** An initial mid–term snapshot (2026–07) contained only six runs under a single condition and is *not* used here; it has been fully superseded by the 270–cell sweep. A first sweep attempt was also discarded after a mid–run reboot corrupted it; the reported run is the clean redo.

6.1.1 Datasets

Table 6.1: The six–dataset panel (all binary classification).

Dataset	Rows	Cols	Source
wdbc	569	31	OpenML 1510
breast-w	699	10	OpenML 15
diabetes	768	9	OpenML 37
Titanic	891	11	curated
spambase	4601	58	OpenML 44
Telco churn	7043	21	curated (IBM)

6.2 Pipeline-Level Results

Table 6.2 gives the headline per–condition result collapsed over all datasets, backends and seeds (90 cells each).

Table 6.2: Per–condition results over all 90 cells each. *Verified yield* = runs that were runnable *and* passed the B2 faithfulness check; *raw execution* = runs that produced a pipeline and a score. B0/B1 have no faithfulness gate, so the two coincide and the verifier is not applicable. Mean score is over executed runs; mean iters over successful runs.

Condition	Verified yield	Raw execution	Mean score	Mean iters
B0 (naive)	90/90 (100.0 %)	90/90 (100.0 %)	0.8432	1.01
B1 (schema)	90/90 (100.0 %)	90/90 (100.0 %)	0.8468	1.06
B2 (meta-feature)	56/90 (**62.2 %**)	63/90 (**70.0 %**)	0.8542*	1.41

* B2's mean is over its 63 executed runs (the 56 verified plus the 7 that executed but failed the faithfulness check); it is still conditional on execution, since the 27 cells that produced no pipeline — often the harder ones — are necessarily excluded. The paired test in Section 6.3.4 is the valid like–for–like comparison.

Table 6.3 separates these outcomes per backend.

Table 6.3: B2 outcomes per backend (30 cells each): verified successes, runs that executed and scored but failed the faithfulness check, other failures, and the resulting raw execution count (raw execution = verified + executed-but-unfaithful).

Backend	Ver.	Exec. unfaithful	Other failures	Raw exec.
gemma4:31b-cloud	24	5	1 syntax_error	29/30
gpt-oss:120b-cloud	24	1	4 runtime_other, 1 api_hallucination	25/30
nemotron-3-nano:30b-cloud	8	1	19 infrastructure, 2 import_error	9/30
Total	**56**	**7**	**27**	**63/90**

The breakdown by backend (Table 6.4) shows the effect is uniform and that B2 is far more expensive. B0 and B1 are perfectly reliable across all three backends; B2's verified yield drops on every backend and is lowest for the smallest model (nemotron–nano: 26.7 % verified, 30.0 % raw execution), whose run–away completion length (26k tokens) trips upstream limits.

Table 6.4: Per–backend × condition: verified success (`ok` = verified yield; raw execution is in Table 6.3), iterations (averaged over all 30 cells, including failed attempts) and mean token counts (over the cells that recorded usage — 30 per row except nemotron–nano B2, where only 14 of 30 did, the rest failing before a usage record was written).

Backend	Cond	ok/30	succ%	iters	prompt tok	compl. tok
gemma4:31b-cloud	B0	30	100.0	1.00	348	627
gemma4:31b-cloud	B1	30	100.0	1.00	1182	611
gemma4:31b-cloud	B2	24	80.0	1.87	9280	2506
gpt-oss:120b-cloud	B0	30	100.0	1.00	369	1338
gpt-oss:120b-cloud	B1	30	100.0	1.03	1170	1346
gpt-oss:120b-cloud	B2	24	80.0	1.70	6449	5330
nemotron-3-nano:30b-cloud	B0	30	100.0	1.03	348	2112
nemotron-3-nano:30b-cloud	B1	30	100.0	1.13	1433	2660
nemotron-3-nano:30b-cloud	B2	8	**26.7**	1.33	7373	**26244**

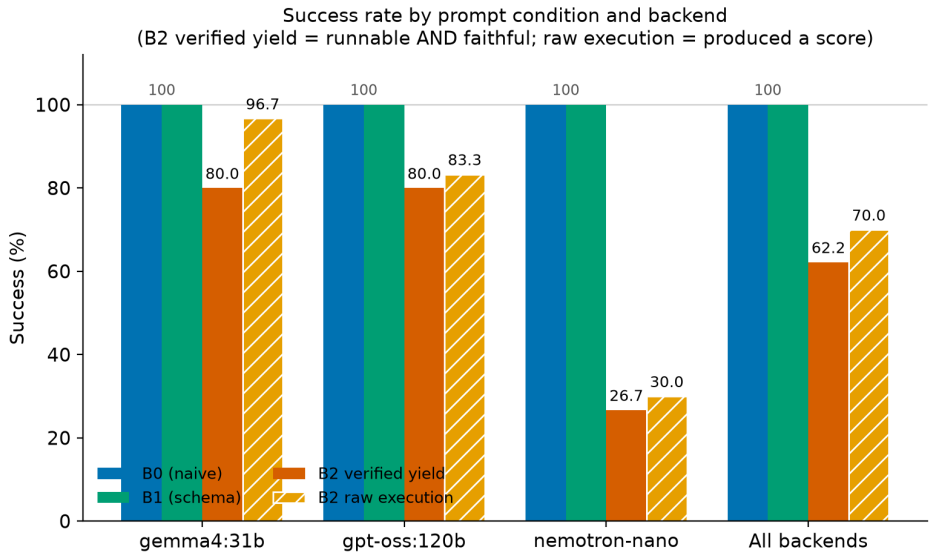


Figure 6.1: Success rate by condition and backend.

Table 6.5 reports mean test score per dataset and condition over the runs that executed. The conditions differ by at most about 0.03 on any dataset (the largest single gap is ≈ 0.025, on Telco and Titanic); meta–features do not materially shift pipeline quality.

Table 6.5: Mean `balanced_accuracy` per dataset × condition over *executed* runs; `n` is the number that executed out of 15 (for B2 this includes runs that produced a score but failed the faithfulness check).

		B0		B1		B2	
Dataset	**rows**	n	score	n	score	n	score
wdbc	569	15	0.9618	15	0.9673	13	0.9706
breast-w	699	15	0.9524	15	0.9492	11	0.9542
diabetes	768	15	0.7215	15	0.7172	12	0.7130
Titanic	891	15	0.7874	15	0.7898	6	0.7670
spambase	4601	15	0.9352	15	0.9313	12	0.9348
Telco churn	7043	15	0.7009	15	0.7259	9	0.7028

6.3 Evaluation of the Research Questions

6.3.1 RQ1 — Representation

The implemented B2 pipeline lifts each plan into a structured reasoning trace (a list of typed `Decision` claims) that is *mechanically* comparable with the generated code. The 63 B2 runs that executed produced a parseable trace that the verifier consumed without a human in the loop; the 27 that failed before producing any output (syntax, import, runtime, API or infrastructure errors) stored a null trace and were not verifiable. This is preliminary support for RQ1 *at the level of the*

implemented representation (presence + value grounding); the richer graph–based PIR of Chapter 3 remains future work, so full coverage of ordering and dataflow claims is not yet demonstrated.

6.3.2 RQ2 — Discrimination (and H1, H2)

This is the project's central claim and it is supported on real data. The verifier flagged **seven** B2 runs as `reasoning_unfaithful`. *All seven executed successfully and produced good scores* (0.717, 0.721, 0.722, 0.925, 0.942, 0.944, 0.969) — yet their stated reasoning did not match their code. An execution–only or performance–only check would have **passed all seven**; the mechanical check rejected them. This is a direct, if small, confirmation of **H1** (execution success does not imply alignment) and preliminary support for **H2** (the mechanical check discriminates where execution cannot).

These seven are counted among the 34 unsuccessful B2 cells, not among the 56 successes: a B2 cell is scored *successful* only if it produces a runnable pipeline *and* passes the faithfulness check. What sets the seven apart from the other 27 failures (syntax, import, runtime, API and infrastructure errors) is that they *ran and produced a good score* — which is exactly why an execution– or performance–only check would have accepted them. Counting them as executions, raw B2 execution is 63/90 = 70.0%, of which the verifier accepted 56 (62.2% verified yield).

6.3.3 RQ3 — Categories

Only B2 produced failures; B0 and B1 were 100 % runnable. The observed failure categories are shown in Table 6.6. The implemented verifier emits a single misalignment bucket, `reasoning_unfaithful`; the finer taxonomy of Chapter 3 (missing operation, wrong parameter, wrong order, ...) is designed but not yet emitted, so RQ3 is answered only partially — misalignment *does* occur in practice (7 cases), but its sub–categorisation is future work.

Table 6.6: Failure categories for the 34 B2 cells that did not meet the end–to–end success definition (B0/B1 had none). `reasoning_unfaithful` marks a run that executed and scored but was rejected by the verifier, not an execution failure.

Backend	Category	n
gemma4:31b-cloud	reasoning_unfaithful	5
gemma4:31b-cloud	syntax_error	1
gpt-oss:120b-cloud	runtime_other	4
gpt-oss:120b-cloud	api_hallucination	1
gpt-oss:120b-cloud	reasoning_unfaithful	1
nemotron-3-nano:30b-cloud	infrastructure	19
nemotron-3-nano:30b-cloud	import_error	2
nemotron-3-nano:30b-cloud	reasoning_unfaithful	1

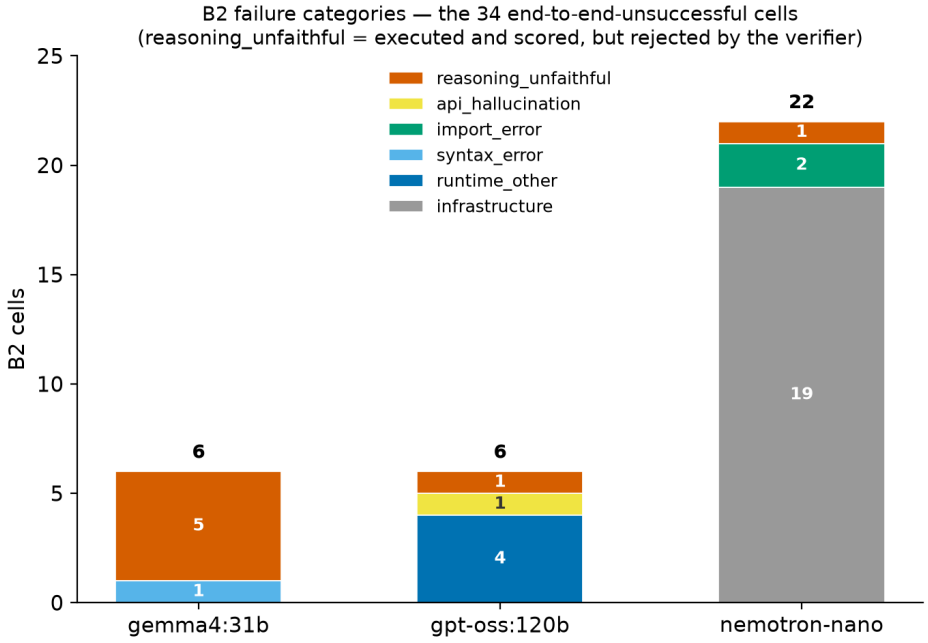


Figure 6.2: Distribution of B2 failure categories.

6.3.4 RQ4 — Variation and Predictive Score

Variation. Faithfulness and reliability vary clearly by backend: all seven unfaithful flags concentrate as gemma 5, gpt-oss 1, nemotron 1, and B2 success ranges from 80 % (the two larger models) to 26.7 % (nemotron–nano). This is preliminary support for RQ4's claim that behaviour varies with the LLM; variation by pipeline complexity (H4) is *not* tested here, as the panel was not stratified by operation count.

Predictive score. A paired Wilcoxon signed–rank test on `test_score`, matched on each (dataset, seed, backend) cell where both conditions produced a

score — so B2 contributes every executed run, whether or not the verifier accepted it (Table 6.7) — shows **no statistically significant difference** for any pair (Bonferroni–adjusted $\alpha = 0.0167$). Meta–feature prompting does not improve predictive quality over naive or schema prompting at this sample size.

Table 6.7: Paired Wilcoxon signed–rank on test score, over matched cells where both conditions produced a score. **n** is the number of *discordant* pairs (those with a non–zero score difference) used by the signed–rank test; the remaining matched pairs scored identically. Median Δ (second condition minus first) is taken over those discordant pairs. $\mathbf{r} = |z|/\sqrt{n}$ is the rank effect size; all three are small (≤ 0.21). B2 includes every run that produced a score, whether or not the verifier accepted it.

Pair	n	median Δ	z	p	r	verdict
B0 vs B1	64	+0.0044	−1.669	0.095	0.21	n.s.
B0 vs B2	60	+0.0000	−0.214	0.831	0.03	n.s.
B1 vs B2	58	+0.0007	−0.159	0.874	0.02	n.s.

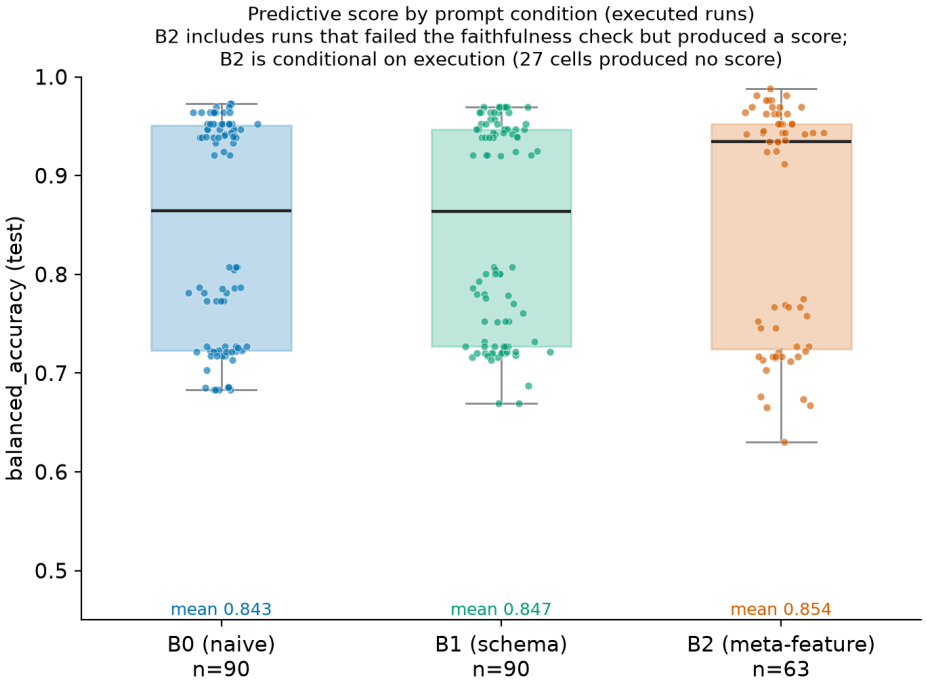


Figure 6.3: Predictive score by condition.

6.3.5 RQ5 — Localisation

The implemented verifier localises a failure to the *specific decision(s)* that did not verify (it records, per decision, whether the action was present and whether the value matched, with a human–readable reason). It does not yet localise to a *source span* in the code, which requires the source–located PIR of Chapter 3. RQ5 is therefore supported at claim level and pending at span level.

6.4 Hypothesis Summary

Table 6.8: Status of the four hypotheses against the 270–cell run.

H	Statement	Status
H1	Execution success does not imply alignment	**Supported** (7 executing, scoring runs flagged unfaithful)
H2	Mechanical alignment discriminates better than execution success	Preliminary support
H3	Mismatches form stable AutoML–specific categories	Pending (full taxonomy not yet emitted)
H4	Mismatch probability grows with pipeline complexity	Pending (no complexity stratification)

6.5 Comparison with Baselines (Planned)

Chapter 3 specifies a comparison of the mechanical verifier against four baselines — execution–only, performance–only, LLM–judge and keyword/API — on a seeded–mutation corpus with human–annotated targets. That corpus and its annotations have not yet been built, so the quantitative comparison (false–alignment rate, category macro–F1, localisation IoU) is **future work**; Table 6.9 records its pre–committed structure.

Table 6.9: Structure of the planned baseline comparison. Entries are left blank pending the seeded–mutation corpus.

Method	Coverage	S_{align}	CMR ↓	Loc. macro–F1
Execution–only	–	–	–	–
Performance–only	–	–	–	–
LLM judge	–	–	–	–
Keyword / API match	–	–	–	–
Mechanical verifier (proposed)	–	–	–	–

One claim must be stated carefully. It is *definitionally* true that an execution– or performance–only baseline, which never reads the plan, cannot flag a reasoning–code misalignment that is compatible with running code and a good score — the seven runs of Section 6.3.2 are concrete instances. But a precise *false–alignment rate* for such a baseline is a quantity to be *measured* on the

annotated corpus, not asserted as 100 %; the earlier draft's blanket "100 % false alignment" claim has been removed as unsupported.

6.6 Discussion

A useful negative result. Across three backends and six datasets, injecting meta–features and decision rules into the prompt (B2) did not improve predictive score (Table 6.7) and markedly reduced reliability (100 % → 62.2 % verified yield, 70.0 % raw execution) at roughly 5–27× the prompt–token cost (about 5–8× over B1 and 18–27× over B0). For this family of models and tabular tasks, the schema signal (B1) already captures what helps; the extra meta–feature payload appears mainly to add failure surface rather than useful signal. This is a clean, reportable observation, not a disappointment — it tells a practitioner they can stop paying the meta–feature cost. That the larger prompt *causes* the extra failures is a plausible reading, not something this design isolates.

The verifier earns its place. The same B2 machinery that hurts reliability is what makes faithfulness *checkable*. The seven unfaithful detections are exactly the cases execution– and performance–based evaluation cannot see, which is the thesis of this project demonstrated end to end on real LLM output.

A caveat on the trace. Because the implemented B2 trace is printed by the generated code at runtime, it is a post–hoc self–report rather than an independently captured plan. The verifier can therefore catch a trace that contradicts its own code, but a model that writes mismatched code *and* a matching (equally wrong) trace would pass. Independent pre–code reasoning capture (`B2R`) is the key next step.

6.7 Threats to Validity

External validity. Three Ollama backends and six binary classification datasets are a small sample; results may not generalise to other model families, to regression or multiclass tasks, or to high–cardinality/time–series data.

Construct validity. B0/B1/B2 are three specific prompts, not the best possible prompt in each class; a different B2 wording could behave differently. The faithfulness verifier checks presence and value grounding only — it does not yet check parameters, ordering or columns, so "faithful" here is a necessary, not sufficient, condition for full alignment. Moreover, the presence check is satisfied whenever the cited action *symbol* appears anywhere in the code — including in an `import` that is never instantiated or applied — so it does not prove the operation was actually used; the boolean verdict can admit false positives, and a PASS means

only that the implemented checks passed, not that the pipeline is semantically faithful.

Internal validity. The implemented verifier treats an *uncheckable* value as not–contradicted, so a decision citing no numeric meta–feature cannot currently be refuted; the proposed three–valued verdict would mark it UNVERIFIABLE. Survivorship bias affects any score comparison that conditions on B2 success, which is why the paired test restricts to matched, both–succeeded cells.

Conclusion validity. Five seeds and a single seed family (3001–3005) bound the statistical power; the non–significant score differences should be read as "no evidence of an effect" at this sample size, not as proof of exact equality.

Reproducibility. Model identifiers, seeds, generated code and JSON traces are all persisted, and the analysis layer reads only stored rows, so every table here is reproducible from the archived run.

6.8 Summary

On a real 270–cell sweep, the naive and schema prompt conditions were perfectly reliable while the meta–feature condition was markedly less reliable (62.2 % verified yield, 70.0 % raw execution) and far more expensive (roughly 5–27× the prompt tokens) without improving predictive score (paired Wilcoxon over executed runs, all $p > 0.09$). The implemented mechanical verifier flagged seven pipelines that executed and scored well but whose reasoning did not match their code — supporting H1 and giving preliminary support to H2, the project's core hypotheses. The representation (RQ1), discrimination (RQ2) and claim–level localisation (RQ5) questions are answered at the level of the implemented main–branch verifier (presence + value grounding); the full taxonomy (RQ3/H3), complexity variation (H4) and the quantitative baseline comparison remain future work pending the richer verifier and a seeded–mutation corpus.

Chapter 7

Conclusion and Future Scope

7.1 Conclusion

The project set out to answer a single operational question: when an LLM–driven AutoML system presents a natural–language or semi–structured plan and then emits a Python pipeline, does the emitted code mechanically implement the plan it claims to realise? Existing evaluations check that the code compiles, that it runs, that it does not raise exceptions and that it reaches an acceptable score on some metric. None of these checks answers the alignment question, which is strictly stronger and which decides whether the documentation, approvals and deployment gates that users see actually describe the pipeline that is running.

To address this, the project has:

- defined *reasoning–code alignment* as a stand–alone verification problem that is cleanly decoupled from reasoning correctness, program correctness, execution success and predictive performance;
- designed a Reasoning Intermediate Representation of atomic typed claims, a Program Intermediate Representation that fuses abstract syntax, dataflow and library–level semantics, and a mechanical mapping between the two;
- specified coverage, weighted alignment, critical mismatch and localisation measures, together with an AutoML–specific mismatch taxonomy covering missing, contradictory, model–level, parameter–level, ordering, evaluation, data–handling, constraint, unsupported, dead–reasoning and extraneous categories;
- proposed an evaluation protocol against execution–only, performance–only, LLM–judge and keyword/API baselines on natural LLM outputs and on seeded–mutation corpora with human–annotated targets, with a statistical design that reports false–alignment rates, macro–F1 of category detection, and localisation accuracy with appropriate confidence intervals; and
- engineered a running end–to–end implementation — five FastAPI services and a Streamlit front–end, a Redis Queue worker, a configurable LLM runner (Ollama in the reported runs), a bounded subprocess executor, a first faithfulness verifier, a PostgreSQL database with local–filesystem artefact storage and RQ1–RQ5 analysis handlers; and

- evaluated that implementation on a real 270–cell sweep, where the naive and schema prompt conditions were perfectly reliable, the meta–feature condition was markedly less reliable and more expensive without improving predictive score, and the mechanical verifier flagged seven pipelines that executed and scored well but whose reasoning did not match their code.

The objectives stated in Chapter 1 are therefore met at three distinct levels, which the report keeps separate. *Design and formalisation* of the full reasoning–code alignment framework is complete. *Software* exists and runs for the pipeline and a first faithfulness verifier (action presence and meta–feature value grounding) on `main`. *Empirical evidence* exists for the prompting study and for the core hypothesis that execution success does not imply alignment (H1), with preliminary support for discrimination (H2). What remains is not design but construction and measurement: the graph–based verifier, independent pre–code reasoning capture, and the seeded–mutation corpus needed to quantify the baseline comparison (H3, H4 and the full RQ2/RQ5 metrics). The project thus delivers a validated foundation and a precise, bounded agenda for the remaining work, rather than a completed verifier presented as finished.

Taken together, these contributions position the proposed verification layer as a complement, not a replacement, for existing AutoML evaluation. Where execution evidence tells us that a pipeline *can* succeed, reasoning–code alignment tells us whether it has succeeded for the *reason* it was supposed to.

7.2 Limitations

The present system has several limitations that bound the strength of its claims.

Plan expressiveness. Natural–language plans may be vague, incomplete or even impossible to execute. Structured claims decrease ambiguity, but they also change the reasoning distribution and ignore helpful context. A learnt RIR extractor may itself reflect the extractor's interpretation of the plan rather than the LLM's intent.

Static analysis of Python. The arbitrary behaviour of Python cannot be determined by static analysis alone, because of reflection, dynamic imports, metaprogramming, services, callbacks and notebook state limiting graph recovery. Runtime traces only exercise paths that were taken. General semantic equivalence is undecidable; bounded evidence via behavioural probes necessarily leaves such cases as UNVERIFIABLE.

Ontology and ranking. The proposed verifier relies on heuristic operator–ontology mapping and candidate ranking, which lean towards common

libraries and may not cover custom code well. High alignment is also not a guarantee of quality: a pipeline can be faithful to its plan yet the plan itself be poor, the data biased or the model poorly calibrated. Finally, the emitted reasoning does not reveal the model's internal computation; the project checks artefact–to–artefact conformance, not mechanistic faithfulness.

Post–hoc reasoning trace. The B2 reasoning trace is printed by the generated code at run time, so it is a post–hoc self–report rather than a plan captured before the code. The verifier can catch a trace that contradicts its own code, but not a case where the code and the trace are wrong in the same way (for example, both naming `StandardScaler` when the dataset called for `RobustScaler`). Independent pre–code reasoning capture (`B2R`) is designed but not implemented, and is the single most important next step.

Scope. The first version of the system targets compositional scikit–learn tabular pipelines. Deep–learning, distributed training, multi–modal workflows and notebooks with hidden state remain outside scope.

LLM supply–chain. Hosted models, prompts, APIs and libraries change over time; alignment numbers must be interpreted alongside the model identifiers, dates, temperatures, seeds and package versions recorded at run time.

7.3 Future Scope

The project opens several promising directions.

Deeper program semantics. Symbolic execution and SMT constraints for parameters and branches, blending static checks with runtime monitors and causal traces, can push the verifier past the current static–analysis bound.

Typed pipeline IRs. Building typed pipeline IRs for proof–carrying AutoML would turn the current best–effort witness mapping into a certified contract between the planner and the coder.

Broader domains. Extensions comprise deep–learning and multimodal pipelines, contracts between agents, deployment–time checks and certified generation.

Repair loops. An optimising repair loop can take the localised mismatch and ask a coder to patch only the diverged subgraph. This must respect the rule set fixed during evaluation; high–impact UNVERIFIABLE claims should still be subject to human approval.

Integration into AutoML products. The verifier can be offered as a drop–in library inside existing AutoML products so that any pipeline that is deployed can be audited claim by claim against the plan that was approved. Such mechanisms would not, by definition, make AutoML correct, but would at least al-

low for one promise to be audited: that the code executed implements the pipeline claimed by the system.

7.4 Final Remarks

Execution tests can miss a verification problem with LLM–driven AutoML. A system may proclaim a consistent strategy for preprocessing, model choice, ordering or evaluation, produce code that runs and gives a passable score, and still not implement the plan it presented; reasoning–code alignment cannot be replaced by execution correctness and predictive performance. This project formalised alignment, proposed the reasoning and program intermediate representations, defined the mechanical verdicts, metrics, taxonomy and baselines, and — crucially — built a running system and measured it. On that first 270–cell run, seven pipelines that executed and scored well were caught contradicting their own reasoning: direct evidence that the divergence is real and mechanically detectable. The open empirical question is now narrower and sharper — how *often* successful AutoML code differs from its plan across models, tasks and complexity, and how precisely the divergence can be localised — and the software, protocol and first measurements are in place to answer it.

Bibliography

[1] M. Feurer, K. Eggensperger, S. Falkner, M. Lindauer, and F. Hutter, "Auto-Sklearn 2.0: Hands-free AutoML via meta-learning," *J. Mach. Learn. Res.*, vol. 23, no. 261, pp. 1–61, 2022.

[2] P. Gijsbers, M. L. Bueno, S. Coors, E. LeDell, S. Poirier, J. Thomas, B. Bischl, and J. Vanschoren, "AMLB: An AutoML benchmark," *J. Mach. Learn. Res.*, vol. 25, no. 101, pp. 1–65, 2024.

[3] N. Hollmann, S. Müller, and F. Hutter, "Large language models for automated data science: Introducing CAAFE for context-aware automated feature engineering," in *Advances in Neural Information Processing Systems*, vol. 36, 2023, pp. 44 753–44 775.

[4] S. Hong, Y. Lin, B. Liu, B. Wu, D. Li, J. Chen, J. Zhang, J. Wang, L. Zhang, W. Zhan, M. Zhuge *et al.*, "Data interpreter: An LLM agent for data science," *arXiv preprint arXiv:2402.18679*, 2024.

[5] S. Guo, C. Deng, Y. Wen, H. Chen, Y. Chang, and J. Wang, "DS-Agent: Automated data science by empowering large language models with case-based reasoning," in *Proc. 41st Int. Conf. Machine Learning*, 2024.

[6] Y. Chi, Y. Lin, S. Hong, D. Pan, Y. Fei, G. Mei, B. Liu, T. Pang, J. Kwok, C. Zhang *et al.*, "SELA: Tree-search enhanced LLM agents for automated machine learning," *arXiv preprint arXiv:2410.17238*, 2024.

[7] P. Trirat, W. Jeong, and S. J. Hwang, "AutoML-Agent: A multi-agent LLM framework for full-pipeline AutoML," in *Proc. 42nd Int. Conf. Machine Learning*, ser. PMLR, vol. 267, 2025.

[8] Y. Gu, H. You, Y. Cao, M. Yu, H. Fan, and S. Qian, "Large language models for constructing and optimizing machine learning workflows: A survey," *arXiv preprint arXiv:2411.10478*, 2024.

[9] A. Madaan, N. Tandon, P. Gupta, S. Hallinan, L. Gao, S. Wiegreffe, U. Alon, N. Dziri, S. Prabhumoye, Y. Yang *et al.*, "Self-Refine: Iterative refinement with self-feedback," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[10] Z. Gou, Z. Shao, Y. Gong, Y. Shen, Y. Yang, N. Duan, and W. Chen, "CRITIC: Large language models can self-correct with tool-interactive critiquing," in *Proc. Int. Conf. Learning Representations*, 2024.

[11] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[12] J. Austin, A. Odena, M. Nye, M. Bosma, H. Michalewski, D. Dohan, E. Jiang, C. J. Cai, M. Terry, Q. Le, and C. Sutton, "Program synthesis with large language models," *arXiv preprint arXiv:2108.07732*, 2021.

[13] H. Pearce, B. Tan, B. Ahmad, R. Karri, and B. Dolan-Gavitt, "Asleep at the keyboard? Assessing the security of GitHub Copilot code contributions," in *Proc. IEEE Symp. Security and Privacy*, 2022, pp. 754–768.

[14] J. Jiang, F. Wang, J. Shen, S. Kim, and S. Kim, "A survey on large language models for code generation," *ACM Computing Surveys*, 2026, preprint arXiv:2406.00515.

[15] J. Shen, X. Peng, and D. Owen, "Failure-aware enhancements for large language model (LLM) code generation: An empirical study on decision framework," in *Proc. IEEE Int. Conf. Software Analysis, Evolution and Reengineering (SANER)*, 2026.

[16] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems*, vol. 35, 2022.

[17] M. Turpin, J. Michael, E. Perez, and S. R. Bowman, "Language models don't always say what they think: Unfaithful explanations in chain-of-thought prompting," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[18] R. J. Young, "Lie to me: How faithful is chain-of-thought reasoning in reasoning models?" *arXiv preprint arXiv:2603.22582*, 2026.

[19] A. Jacovi, Y. Bitton, B. Bohnet, J. Herzig, O. Honovich, M. Tseng, M. Collins, R. Aharoni, and M. Geva, "A chain-of-thought is as strong as its weakest link: A benchmark for verifiers of reasoning chains," in *Proc. 62nd Annu. Meeting of the Association for Computational Linguistics*, 2024, pp. 4615–4634.

[20] H. Lightman, V. Kosaraju, Y. Burda, H. Edwards, B. Baker, T. Lee, J. Leike, J. Schulman, I. Sutskever, and K. Cobbe, "Let's verify step by step," in *Proc. Int. Conf. Learning Representations*, 2024.

[21] E. Perrier, "Typed chain-of-thought: A curry–howard framework for verifying LLM reasoning," *arXiv preprint arXiv:2510.01069*, 2025.

[22] Z. Zhao, Y. Koishekenov, X. Yang, N. Murray, and N. Cancedda, "Verifying chain-of-thought reasoning via its computational graph," in *Proc. Int. Conf. Learning Representations (ICLR)*, 2026, oral.

[23] F. Pedregosa, G. Varoquaux, A. Gramfort, V. Michel, B. Thirion, O. Grisel, M. Blondel, P. Prettenhofer, R. Weiss, V. Dubourg *et al.*, "Scikit-learn: Machine learning in Python," *J. Mach. Learn. Res.*, vol. 12, pp. 2825–2830, 2011.

[24] N. Abhyankar, P. Shojaee, and C. K. Reddy, "LLM-FE: Automated feature engineering for tabular data with LLMs as evolutionary optimizers," *Trans. Mach. Learn. Res.*, 2026.

[25] S. Fathollahzadeh, E. Mansour, and M. Boehm, "CatDB: Data-catalog-guided LLM-based generation of data-centric ML pipelines," *Proc. VLDB Endowment*, vol. 18, no. 8, 2025.

[26] K. Ni, L. Cao, J. He, X. Zhang, J. Jin, J. Dong, and Y. Yu, "FlowPipe: LLM-enhanced conditional generative flow networks for data preparation pipeline construction," arXiv preprint arXiv:2606.24679, 2026, preprint; formal publication status not independently verified at the time of writing.

[27] C. A. R. Hoare, "An axiomatic basis for computer programming," *Commun. ACM*, vol. 12, no. 10, pp. 576–580, 1969.

[28] P. Cousot and R. Cousot, "Abstract interpretation: A unified lattice model for static analysis of programs by construction or approximation of fixpoints," in *Proc. 4th ACM SIGACT-SIGPLAN Symp. Principles of Programming Languages*, 1977, pp. 238–252.

[29] E. M. Clarke, O. Grumberg, and D. A. Peled, *Model Checking*. Cambridge, MA, USA: MIT Press, 1999.

[30] G. Tyen, H. Mansoor, V. Cârbune, Y. P. Chen, and T. Mak, "LLMs cannot find reasoning errors but can correct them given the error location," in *Findings of the Association for Computational Linguistics: ACL*, 2024, pp. 13 894–13 908.

[31] J. Li, G. Li, Y. Li, and Z. Jin, "Structured chain-of-thought prompting for code generation," *ACM Trans. Softw. Eng. Methodol.*, vol. 34, no. 2, pp. 1–23, 2025.

Chapter A

Appendix

This appendix presents additional academic, research, innovation, social-impact, and project-related contributions made by the students during the development of the major project.

Author note: the sections that follow with no project evidence — Patent/Copyright, Data Contribution, Research Grant, Commercialization (Startup / Technology Transfer / Industry Collaboration), Research Conclave, the SDG mapping table, and the CO–PO and CO–PSO mapping tables — each require either the official departmental information or a "Not Applicable" marking per the prescribed VESIT format. No patents, grants, commercial activity, industry partnerships, conclave participation, or SDG/CO mappings have been invented; complete or mark each section before submission.

A.1 Publication

A research paper based on this project has been prepared for submission.

A.1.1 Publication Details

- **Title of the paper:** Mechanical Verification of Reasoning–Code Alignment in LLM–Driven AutoML Pipelines
- **Authors:** Kinnshuk Bhaduri, Samarth Adsare, Ojas Godambe, and Sarvesh Rakte
- **Journal / Conference:** (to be filled in on acceptance)
- **Publisher:** (to be filled in on acceptance)
- **Date of publication / acceptance:** (to be filled in on acceptance)
- **DOI / Paper ID / URL:** (to be filled in on acceptance)

A.1.2 Publication Evidence

The following items will be attached as evidence:

- Final submitted / published paper (PDF).

- Acceptance letter, if applicable.
- Conference certificate, if applicable.
- DOI page, if applicable.
- Conference / journal proceedings page, if applicable.

A.2 Patent / Copyright

Provide details of any patent application, granted patent, copyright registration, or other intellectual property generated from the project.

A.2.1 Intellectual Property Details

- Title:
- Type: Patent / Copyright
- Applicant(s):
- Inventor(s) / Author(s):
- Application / Registration Number:
- Date of Filing / Registration:
- Status:

A.2.2 Evidence

Attach relevant patent or copyright documentation.

A.3 Open Source Contribution

The project itself is released as open–source software. The complete stack is split across eight repositories under the project's GitHub organisation, each dual–licensed under the permissive **MIT OR Apache–2.0** terms (declared via an `SPDX-License-Identifier: MIT OR Apache-2.0` header in the source files).

- **Project:** LLM–Driven AutoML with reasoning–code alignment verification.
- **Repositories** (GitHub organisation `Major-Proj-AutoML`):

  - https://github.com/Major-Proj-AutoML/automl-reusables (shared library)
  - https://github.com/Major-Proj-AutoML/automl-data-service
  - https://github.com/Major-Proj-AutoML/automl-metafeatures-service
  - https://github.com/Major-Proj-AutoML/automl-generation-service
  - https://github.com/Major-Proj-AutoML/automl-analysis-service
  - https://github.com/Major-Proj-AutoML/automl-gateway
  - https://github.com/Major-Proj-AutoML/automl-streamlit (front-end)
  - https://github.com/Major-Proj-AutoML/automl-infra (orchestration)

- **Platform:** GitHub.
- **Student Contributors:** Samarth Adsare, Kinnshuk Bhaduri, Ojas Godambe, Sarvesh Rakte.
- **Licence:** MIT OR Apache–2.0 (dual).

A.3.1 Evidence

These eight repositories are authored and maintained by the project team — they are the project's own code, not third–party projects the team merely depends on. The public repository URLs listed above, the SPDX licence headers, and the per–repository commit history are the contribution evidence. (A DOI for an archived release is pending.)

A.4 Data Contribution

Describe any dataset created, curated, annotated, cleaned, published, or contributed by the students.

- Dataset Name:
- Dataset Description:
- Source:
- Number of Records / Samples:
- Data Collection Method:

- Annotation / Preprocessing Performed:
- Dataset Repository / URL:

A.4.1 Evidence

Include dataset documentation, repository screenshots, data cards, or relevant links.

A.5 Research Grant Proposal

Mention any research grant proposal prepared or submitted based on the project.

- Grant / Funding Scheme:
- Proposal Title:
- Student Contributor(s):
- Faculty Mentor:
- Funding Agency:
- Proposed Amount:
- Submission Date:
- Status:

A.5.1 Evidence

Attach proposal documents, acknowledgement of submission, or other relevant evidence.

A.6 Research Commercialization Efforts

Describe efforts undertaken to translate the project or research outcomes into practical or commercial applications.

A.6.1 Startup Formation

If applicable, describe the steps taken towards startup formation.

- Startup / Venture Name:

- Student Founders:
- Problem Addressed:
- Proposed Solution:
- Current Stage:

A.6.2 Technology Transfer

Describe any technology transfer activity associated with the project.

- Technology / Solution:
- Receiving Organization:
- Nature of Transfer:
- Date:
- Current Status:

A.6.3 Industry Collaboration

Describe any collaboration with an industry partner.

- Industry Partner:
- Nature of Collaboration:
- Student Contributors:
- Project / Activity:
- Duration:
- Outcome:

A.6.4 Evidence

Attach MoUs, collaboration letters, technology-transfer documents, industry letters, meeting records, or other relevant evidence.

A.7 Research Conclave

Mention participation, presentation, demonstration, poster presentation, or other contribution made by the students at a Research Conclave.

- Event Name:
- Organizing Institution:
- Date:
- Project / Research Title:
- Student Participant(s):
- Type of Participation:
- Award / Recognition, if any:

A.7.1 Evidence

Attach participation certificates, photographs, presentation certificates, awards, or other supporting documents.

A.8 SDG Goals Achieved

Describe the United Nations Sustainable Development Goals (SDGs) addressed by the project.

Table A.1: SDG Goals Addressed by the Project

SDG	Goal	Contribution of the Project
SDG 1		
SDG 2		
SDG 3		
SDG 4		
SDG 5		
SDG 6		
SDG 7		
SDG 8		
SDG 9		
SDG 10		
SDG 11		
SDG 12		
SDG 13		
SDG 14		
SDG 15		
SDG 16		
SDG 17		

Students should retain only the SDGs that are genuinely relevant to the project and provide a clear justification for each selected goal.

A.9 Course Outcomes, PO and PSO Mapping

This section presents the Course Outcomes (COs), Program Outcomes (POs), Programme Specific Outcomes (PSOs), and their mapping with the major project.

A.9.1 Course Outcomes

The Course Outcomes (COs) of the Major Project are as follows:

CO1: Articulate the nature of science, differentiate between objective and subjective thinking, and situate scientific inquiry within its historical and philosophical context.

CO2: Apply inductive, deductive, and syllogistic reasoning to construct and evaluate valid scientific arguments and identify causal relationships.

CO3: Apply statistical concepts including the Central Limit Theorem, error propagation, confidence intervals, and data visualization techniques to interpret experimental results.

CO4: Design and execute hypothesis testing procedures using Z-test, T-test, and Chi-Square tests, and critically evaluate the validity of scientific claims.

CO5: Construct mathematical models, perform dimensional analysis, and derive functional relationships to support theoretical research.

CO6: Produce well-structured scientific documents, deliver conference presentations, and demonstrate ethical conduct in research and publication.

A.9.2 Project Contribution to Program Outcomes

The major project provides opportunities for students to demonstrate the following Program Outcomes through problem identification, literature review, system design, implementation, experimentation, analysis, documentation, presentation, teamwork, and project management.

Table A.2: Project Contribution to Program Outcomes

PO	Outcome	Evidence from Project
PO1	Engineering Knowledge	Describe how mathematical, scientific, computing, and engineering knowledge is applied in the project.
PO2	Problem Analysis	Describe problem identification, literature review, analysis, and formulation performed during the project.
PO3	Design/Development of Solutions	Describe the design and development of the proposed solution.
PO4	Conduct Investigations	Describe experimentation, modelling, testing, data analysis, and interpretation performed.
PO5	Engineering Tool Usage	Describe the software, frameworks, computational tools, and technologies used.
PO6	The Engineer and The World	Describe societal, environmental, sustainability, health, safety, legal, or cultural considerations.
PO7	Ethics	Describe ethical considerations, responsible data usage, privacy, intellectual property, and compliance.
PO8	Individual and Collaborative Team Work	Describe individual responsibilities, teamwork, coordination, and collaboration.
PO9	Communication	Describe technical documentation, presentations, demonstrations, and communication of project outcomes.
PO10	Project Management and Finance	Describe project planning, scheduling, resource management, budgeting, and decision-making.
PO11	Life-Long Learning	Describe new technologies, independent learning, research, adaptation, and continuous skill development.

A.9.3 Project Contribution to Programme Specific Outcomes

Table A.3: Project Contribution to Programme Specific Outcomes

PSO	Outcome	Evidence from Project
PSO1	Professional Skills	Describe the application of mathematics, computational methods, statistics, artificial intelligence, and data science concepts.
PSO2	Successful Career	Describe the design and implementation of the proposed solution using relevant state-of-the-art AI and Data Science techniques.

CO–PO Mapping

Mapping Scale: 3 – High Contribution, 2 – Moderate Contribution, 1 – Low Contribution, – – No Significant Contribution

Table A.4: CO–PO Mapping

CO / PO	PO1	PO2	PO3	PO4	PO5	PO6	PO7	PO8	PO9	PO10	PO11
CO1											
CO2											
CO3											
CO4											
CO5											
CO6											

CO–PSO Mapping

Mapping Scale: 3 – High Contribution, 2 – Moderate Contribution, 1 – Low Contribution, – – No Significant Contribution

Table A.5: CO–PSO Mapping

CO / PSO	PSO1	PSO2
CO1		
CO2		
CO3		
CO4		
CO5		
CO6		

CO–PO / CO–PSO Mapping Justification

The CO–PO and CO–PSO mappings shall be determined based on the actual contribution of the project towards the attainment of the respective Course Outcomes (COs), Program Outcomes (POs), and Program Specific Outcomes (PSOs).

For each CO, students shall identify all relevant POs and PSOs to which the CO is mapped, specify the strength of each mapping, and provide suitable justification based on evidence from the project.

Mapping Strength

- **3 – High Contribution:** Substantial and direct contribution towards the attainment of the corresponding PO or PSO.
- **2 – Moderate Contribution:** Meaningful but partial contribution towards the attainment of the corresponding PO or PSO.
- **1 – Low Contribution:** Limited or indirect contribution towards the attainment of the corresponding PO or PSO.
- **– – No Significant Contribution:** No meaningful contribution towards the corresponding PO or PSO.

Students should map only those POs and PSOs for which there is sufficient evidence from the project. The strength of each mapping shall be assigned independently. Additional POs or PSOs may be included wherever applicable.

CO1

CO1: Articulate the nature of science, differentiate between objective and subjective thinking, and situate scientific inquiry within its historical and philosophical context.

POs mapped: _____
Strength of mapping: _____
Justification:

PSOs mapped: _____
Strength of mapping: _____
Justification:

CO2

CO2: Apply inductive, deductive, and syllogistic reasoning to construct and evaluate valid scientific arguments and identify causal relationships.

POs mapped: _____
Strength of mapping: _____
Justification:

PSOs mapped: _____
Strength of mapping: _____
Justification:

CO3

CO3: Apply statistical concepts including the Central Limit Theorem, error propagation, confidence intervals, and data visualization techniques to interpret experimental results.

POs mapped: _____
Strength of mapping: _____
Justification:

PSOs mapped: _____
Strength of mapping: _____
Justification:

CO4

CO4: Design and execute hypothesis testing procedures using Z-test, T-test, and Chi-Square tests, and critically evaluate the validity of scientific claims.

POs mapped: _____
Strength of mapping: _____
Justification:

PSOs mapped: _____
Strength of mapping: _____
Justification:

CO5

CO5: Construct mathematical models, perform dimensional analysis, and derive functional relationships to support theoretical research.

POs mapped: _____
Strength of mapping: _____
Justification:

PSOs mapped: _____
Strength of mapping: _____
Justification:

CO6

CO6: Produce well-structured scientific documents, deliver conference presentations, and demonstrate ethical conduct in research and publication.

POs mapped: _____
Strength of mapping: _____
Justification:

PSOs mapped: _____
Strength of mapping: _____
Justification:

Guide Verification

I hereby verify that the CO–PO and CO–PSO mappings of this project have been reviewed and corrected, wherever necessary, to accurately reflect the contribution of the project towards the prescribed Course Outcomes, Program Outcomes, and Program Specific Outcomes.

_____

Mr. Amit Singh

Project Guide

Date: _____

A.10 Additional Evidence

Include any other relevant evidence, certificates, photographs, awards, presentations, technical reports, or supporting documents associated with the major project.